

TUGAS AKHIR - EE234899

NAVIGASI ROBOT BERBASIS LAYERED COSTMAP UNTUK PERGERAKAN ANTAR LANTAI PADA LINGKUNGAN INDOOR PASCA GEMPA BUMI

BAGAS SURYA WIRAWAN

NRP 5022221026

Dosen Pembimbing

Dr. Ir. Djoko Purwanto, M.Eng.

NIP 196512111990021002

Fajar Budiman, S.T., M.Sc.

NIP 198607072014041001

Program Studi Strata 1 (S1) Teknik Elektro

Departemen Teknik Elektro

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026



TUGAS AKHIR - EE234899

**NAVIGASI ROBOT BERBASIS LAYERED COSTMAP
UNTUK PERGERAKAN ANTAR LANTAI PADA
LINGKUNGAN INDOOR PASCA GEMPA BUMI**

BAGAS SURYA WIRAWAN

NRP 5022221026

Dosen Pembimbing

Dr. Ir. Djoko Purwanto, M.Eng.

NIP 196512111990021002

Fajar Budiman, S.T., M.Sc.

NIP 198607072014041001

Program Studi Strata 1 (S1) Teknik Elektro

Departemen Teknik Elektro

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026

[Halaman ini sengaja dikosongkan]



FINAL PROJECT - EE234899

***ROBOT NAVIGATION USING LAYERED COSTMAP FOR
INTER-FLOOR MOVEMENT IN POST-EARTHQUAKE
INDOOR ENVIRONMENTS***

BAGAS SURYA WIRAWAN

NRP 5022221026

Advisor

Dr. Ir. Djoko Purwanto, M.Eng.

NIP 196512111990021002

Fajar Budiman, S.T., M.Sc.

NIP 198607072014041001

Undergraduate Study Program of Electrical Engineering

Department of Electrical Engineering

Faculty of Intelligent Electrical and Informatics Technology

Sepuluh Nopember Institute of Technology

Surabaya

2026

[Halaman ini sengaja dikosongkan]

ABSTRAK

Nama Mahasiswa : BAGAS SURYA WIRAWAN
Judul Tugas Akhir : NAVIGASI ROBOT BERBASIS LAYERED COSTMAP UNTUK PERGERAKAN ANTAR LANTAI PADA LINGKUNGAN INDOOR PASCA GEMPA BUMI
Pembimbing : 1. Dr. Ir. Djoko Purwanto, M.Eng.
2. Fajar Budiman, S.T., M.Sc.

Indonesia merupakan salah satu negara dengan frekuensi bencana gempa bumi yang tinggi. Gempa bumi menyebabkan perubahan struktural yang signifikan pada lingkungan indoor, terutama pada bangunan bertingkat. Robot otonom dapat berperan mengeksplorasi area indoor pasca gempa yang tidak aman bagi manusia. Oleh karena itu, robot memerlukan kemampuan navigasi yang mampu bekerja pada lingkungan pasca gempa yang bertingkat. Tugas akhir ini membahas pengembangan metode navigasi menggunakan *layered costmap* supaya robot dapat melintasi lingkungan tersebut. Skema ini merepresentasikan lingkungan bertingkat sebagai kumpulan peta *region* berbentuk *occupancy grid* yang digabung menjadi satu *map*. Dalam *costmap* berlapis, tiap layer pada *costmap* menyimpan informasi penting berbeda seperti rintangan statis, *obstacle dinamis*, zona inflasi, dan zona transisi antar *map*, yang memungkinkan navigasi antar *region*. Dalam arsitektur navigasi yang dikembangkan, setiap lantai atau tingkat pada lingkungan *indoor* direpresentasikan secara unik oleh satu *map* individual. Pendekatan ini menyederhanakan informasi lingkungan 3D menjadi representasi 2D yang dapat dikelola, sehingga memungkinkan penggunaan sensor *LiDAR* 2D untuk menentukan lingkungan robot. Algoritma navigasi kemudian mengintegrasikan *map* 2D tiap *region* ini menjadi satu kesatuan dengan menggunakan *Costmap* transisi, sehingga memungkinkan robot untuk memahami dan menavigasi lingkungan multi-lantai. Sistem ini memungkinkan robot untuk menavigasi dari satu *region* ke *region* lain dengan (1) lokalisasi posisi asli dengan elemen ICP dan *region switcher*, (2) *Path Planning* yang membuat rute antar *region* menggunakan elemen *path tracker*, dan (3) *Path Tracking* yang menggunakan *DWA Planner* untuk menghasilkan *trajectory* yang diikuti robot. Algoritma navigasi ini diuji coba di lingkungan simulasi Mujoco yang dirancang untuk kondisi normal dan pasca gempa. Hasil ICP menunjukkan error lebih tinggi pada sumbu *y* dan *yaw* pada kondisi lantai tidak rata dibanding normal, namun masih sukses lokalisasi. Kemudian, hasil pengujian *costmap* menunjukkan nilai *cost scaling factor* 100 paling cocok untuk pembuatan *path* pada kondisi *map* yang digunakan. Robot membutuhkan waktu untuk navigasi dari *region* A sampai C selama 127.26 detik.

Kata Kunci: *Navigasi, Costmap, Robot, Antar lantai*

[Halaman ini sengaja dikosongkan]

ABSTRACT

Name : BAGAS SURYA WIRAWAN
Title : ROBOT NAVIGATION USING LAYERED COSTMAP FOR INTER-FLOOR MOVEMENT IN POST-EARTHQUAKE INDOOR ENVIRONMENTS
Advisors : 1. Dr. Ir. Djoko Purwanto, M.Eng.
2. Fajar Budiman, S.T., M.Sc.

Indonesia is a country experiencing high frequency of earthquake disasters, and so often needs to conduct Search and Rescue (SAR) missions in hazardous indoor environments, which pose significant risks to personnel. To mitigate these risks, autonomous robots can play a role in replacing human personnel in certain conditions. To carry out its mission autonomously, a robot needs navigation capabilities that can operate in multi-story post-earthquake environments. This final project proposal discusses the development of a navigation method using a layered costmap to enable robots to traverse such complex environments. This scheme represents multi-story environments as a collection of layered costmaps. Each layer in the costmap stores crucial information such as static obstacles, dynamic obstacles, inflation zones, and transition regions between maps, enabling comprehensive navigation. In the developed navigation architecture, each floor or level in the indoor environment is uniquely represented by an individual map. This approach effectively simplifies 3D environmental information into a manageable 2D representation, thus allowing the use of a 2D Lidar sensor for robot environmental data collection. The navigation algorithm then intelligently integrates and combines these individual 2D maps into a cohesive whole, enabling the robot to comprehensively understand and navigate multi-floor environments. This system allows the robot to seamlessly transition from one map to another, corresponding to its physical position between different floors within the building. This navigation algorithm will be tested in a laboratory environment designed to simulate post-earthquake conditions.

Keywords: Navigation, Costmap, Robot, Inter-floor.

[Halaman ini sengaja dikosongkan]

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena dengan rahmat dan karunia-Nya penulis dapat menyelesaikan penyusunan tugas akhir yang berjudul "NAVIGASI ROBOT BERBASIS LAYERED COSTMAP UNTUK PERGERAKAN ANTAR LANTAI PADA LINGKUNGAN INDOOR PASCA GEMPA BUMI" dengan baik dan tepat waktu.

Surabaya, Juni 2026

BAGAS SURYA WIRAWAN

[Halaman ini sengaja dikosongkan]

DAFTAR ISI

ABSTRAK	i
ABSTRACT	iii
KATA PENGANTAR	v
DAFTAR ISI	vii
DAFTAR GAMBAR	ix
DAFTAR TABEL	xi
1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Batasan Masalah	2
1.4 Tujuan	2
2 TINJAUAN PUSTAKA	5
2.1 Hasil Penelitian/Perancangan Terdahulu	5
2.2 Teori/Konsep Dasar	6
2.2.1 Layered Costmap	6
2.2.2 Lokalisasi Iterative Closest Point (ICP)	6
2.2.3 Path Planning A*	8
2.2.4 Path Tracking Pure Pursuit	8
3 METODOLOGI	9
3.1 Pengolahan Data Map	9
3.1.1 Map Region	9
3.1.2 Map Transisi	11
3.2 Perancangan Sistem Navigasi	13
3.2.1 Lokalisasi Iterative Closest Point (ICP)	14
3.2.2 Costmap	18
3.2.3 Global Planner	21

3.2.4	Local Planner DWA	24
4	HASIL DAN PEMBAHASAN	29
4.1	Hasil dan Analisis	29
4.1.1	Hasil Lokalisasi ICP	29
4.1.2	Hasil Zona Transisi	33
4.1.3	Hasil Global Planner	35
4.1.4	Hasil DWA Planner	40
4.2	Pembahasan	41
5	PENUTUP	43
5.1	Kesimpulan	43
5.2	Saran	43
	DAFTAR PUSTAKA	45
	LAMPIRAN	47
	BIOGRAFI PENULIS	49

DAFTAR GAMBAR

3.1	Lingkungan arena dalam 3D	9
3.2	Map Occupancy Grid masing-masing region	10
3.3	Map Gabungan	10
3.4	Map transisi antar region	11
3.5	Arsitektur sistem navigasi	13
3.6	Diagram alur navigasi	14
4.1	Simulasi di Region A	29
4.2	Error ICP di Region A	30
4.3	Simulasi di Region A dengan rintangan rantai	30
4.4	Error ICP di Region A dengan rintangan rantai	31
4.5	Simulasi di Region C	32
4.6	Error ICP di Region C	32
4.7	Error ICP di Region A dengan data posisi	33
4.8	Radius zona transisi	34
4.9	Relasi path terhadap Cost Scaling Factor di Region A	35
4.10	Semua path di region A. ● CSF 10, ● CSF 20, ● CSF 50, ● CSF 100	35
4.11	Relasi path terhadap Cost Scaling Factor di Region B	36
4.12	Semua path di region B. ● CSF 10, ● CSF 20, ● CSF 50, ● CSF 100	37
4.13	Relasi path terhadap Cost Scaling Factor di Region C	38
4.14	Semua path di region C. ● CSF 10, ● CSF 20, ● CSF 50, ● CSF 100	39

[Halaman ini sengaja dikosongkan]

DAFTAR TABEL

3.1	Daftar Koneksi pasangan	11
3.2	Orientasi Region terhadap Global Frame	11
3.3	Posisi Inisial Robot	12
3.4	Parameter Default ICP	18
3.5	Nilai konstanta costmap	18
3.6	Parameter Default Cost Function	22
4.1	Karakteristik Path di Region A berdasarkan Cost Scaling Factor	35
4.2	Karakteristik Path di Region B berdasarkan Cost Scaling Factor	36
4.3	Karakteristik Path di Region C berdasarkan Cost Scaling Factor	38
4.4	Perbandingan path terhadap ground truth di region A	39
4.5	Perbandingan path terhadap ground truth di region B	40
4.6	Perbandingan path terhadap ground truth di region C	40
4.7	Hasil tracking path	40
4.8	Hasil tracking path kondisi gempa	40

[Halaman ini sengaja dikosongkan]

BAB I

PENDAHULUAN

1.1 Latar Belakang

Indonesia adalah negara yang sering sekali mengalami bencana gempa bumi. Secara geografis, Indonesia terletak di antara 4 lempeng tektonik, yaitu Lempeng Pasifik, Lempeng Eurasia, Lempeng Indo-Australia, dan Lempeng Laut Filipina. Lokasi Indonesia di antara 4 lempeng ini menyebabkan frekuensi terjadinya gempa bumi cukup tinggi. Berdasarkan data dari BMKG, rata-rata Indonesia mengalami 18 gempa bumi tiap hari (Sabtaji, 2020). Pasca gempa bumi, operasi *search and rescue* (SAR) akan dilakukan di daerah yang terdampak gempa bumi untuk menolong korban dan menanggulangi kerusakan. Seringkali petugas SAR mengalami ancaman terhadap keselamatannya dalam melaksanakan operasi SAR.

Robot bisa ikut berperan dalam membantu petugas SAR melaksanakan tugasnya. Robot UAV mampu memberi gambaran medan pada daerah terjadinya bencana, sedangkan robot darat yang *track* atau *quadruped* dapat membantu membawa barang berat dan membantu dalam hal lainnya pada saat area bencana tidak dapat dilewati kendaraan besar. Robot juga dapat melakukan misi yang terlalu berbahaya untuk dilakukan oleh manusia. Sebagai contoh, sebuah robot darat pernah digunakan pasca gempa di Jepang untuk menginspeksi bangunan yang mengalami kerusakan. Mengirim petugas manusia dinilai terlalu berbahaya karena atap bangunan yang hampir roboh, sehingga digunakan robot yang dikendalikan dari jarak jauh (Lin et al., 2022).

Dalam implementasinya, robot darat SAR dapat dikendalikan manual oleh operator atau bisa otonom. Walaupun robot yang dikendalikan operator memiliki algoritma yang lebih sederhana, robot otonom juga memiliki peran pada operasi SAR, terutama pada daerah di mana infrastruktur komunikasinya rusak (Zhang et al., 2023). Untuk bisa melakukan misi otonom di dalam bangunan, robot darat perlu bisa melakukan lokalisasi, menemukan jalur navigasi yang aman, dan mencapai posisi tujuan, pada lingkungan tidak terstruktur, seperti pada lingkungan indoor dengan banyak puing yang disebabkan oleh gempa. Meskipun beberapa robot mampu bernavigasi pada lingkungan tidak terstruktur, navigasi robot darat pada area indoor masih menjadi permasalahan untuk robot otonom.

Di dalam proposal tugas akhir ini, diusulkan sebuah metode untuk mencoba menjawab permasalahan tersebut. Pada implementasi navigasi untuk robot darat otonom, umum digunakan *costmap* 2D untuk menyimpan informasi lingkungan. Metode ini terbatas saat robot perlu mencapai lokasi yang tidak terletak pada bidang yang sama. Untuk membenahi ini, sebuah metode untuk menyederhanakan informasi lingkungan 3D menjadi direpresentasikan oleh beberapa bidang *costmap* 2D atau *layered costmap*. Tiap *map* dapat mewakili sebuah lantai di dalam bangunan atau lingkungan indoor. Sebuah layer dapat juga menunjukkan tangga penghubung antar lantai. Lalu, robot darat otonom dapat bergerak pada salah satu *map* tertentu, kemudian berpindah ke *map* lain sesuai posisinya di bangunan.

1.2 Rumusan Masalah

Berdasarkan latar belakang pada bagian sebelumnya, terdapat beberapa inti masalah yang perlu dibahas. Permasalahan tersebut dijabarkan sebagai berikut:

1. Bagaimana cara untuk merepresentasikan informasi lingkungan yang terdiri dari beberapa lantai menjadi *layered costmap* untuk bisa digunakan pada algoritma navigasi antar lantai?
2. Bagaimana cara membuat algoritma lokalisasi, *path planning*, dan *path tracking* yang bisa bekerja pada algoritma navigasi yang menggunakan *layered costmap*?
3. Apakah dengan menggunakan metode navigasi antarlantai, robot darat otonom dapat melakukan navigasi hingga mencapai tujuan pada arena lab?

1.3 Batasan Masalah

Dalam tugas akhir ini, metode multilayer *costmap* yang dikembangkan tidak dirancang untuk langsung digunakan di kondisi pasca gempa. Hal ini disebabkan oleh beberapa faktor berikut:

1. Pengujian dilakukan pada arena laboratorium yang didesain untuk merekayasa kondisi pasca gempa. Arena dilengkapi dengan rekayasa medan reruntuhan, tanjakan, dan permukaan tidak rata, yang dibuat pada skala lebih kecil.
2. Navigasi dilakukan oleh robot darat hexapod yang melakukan navigasi secara otonom, dengan bergerak dari titik awal menuju tujuan yang telah ditentukan.
3. Algoritma navigasi yang digunakan digunakan pada kondisi arena laboratorium yang sudah dimapping sebelumnya sehingga lokasi rintangan statis sudah diketahui.

1.4 Tujuan

Tugas akhir ini bertujuan untuk:

1. Mengembangkan metode representasi lingkungan menggunakan *layered costmap* untuk menyederhanakan informasi lingkungan yang terdiri dari beberapa lantai, yang dapat digunakan oleh robot otonom untuk navigasi antar lantai.
2. Merancang dan mengimplementasikan algoritma lokalisasi, *path planning*, dan *path tracking*, yang menggunakan *layered costmap* agar robot dapat bernavigasi antar lantai.
3. Melakukan pengujian sistem navigasi berbasis *layered costmap* di lingkungan laboratorium, untuk mengevaluasi efektivitas dan keterbatasan dari pendekatan yang diusulkan sebelum diterapkan pada skenario nyata.

Tugas akhir ini diharapkan dapat memberikan manfaat sebagai berikut:

1. Penelitian ini memberikan kontribusi terhadap pengembangan sistem navigasi robot otonom, khususnya dalam pemrosesan data lingkungan kompleks menjadi representasi yang lebih sederhana dan terstruktur.

2. Metode multilayer *costmap* yang diusulkan dapat menjadi dasar awal untuk navigasi robot di area bertingkat atau tidak rata, seperti bangunan runtuh pasca gempa.
3. Dasar untuk penelitian lanjutan dalam pengembangan sistem robot yang mampu beroperasi secara otonom di area terdampak bencana, dengan mempertimbangkan faktor medan nyata yang kompleks dan tidak terstruktur

[Halaman ini sengaja dikosongkan]

BAB II

TINJAUAN PUSTAKA

2.1 Hasil Penelitian/Perancangan Terdahulu

Penelitian oleh (Macenski et al., 2023) membahas metode *layered costmap*, yaitu sebuah metode untuk menambah lapisan secara sistematis dengan tiap lapisan mampu menyimpan informasi lingkungan tertentu. Tiap lapisan dapat mewakili sumber informasi, algoritma, ataupun hasil berbeda. Setiap siklus update dapat memperbarui informasi pada grid *costmap*. Manfaatnya dengan menggunakan *layered costmap* termasuk pembaruan yang lebih jelas, area pembaruan yang dinamis, proses pembaruan yang teratur, dan konfigurasi yang fleksibel, memungkinkan representasi nilai biaya yang kompleks dan perilaku navigasi yang peka konteks, seperti navigasi sosial. Implementasi ini telah terintegrasi dengan ROS 2 dan diadopsi sebagai algoritma navigasi default.

Studi oleh (Hu et al., 2022) mengusulkan pendekatan navigasi adaptif kemiringan untuk robot bergerak darat, yang bertujuan mengatasi keterbatasan peta biaya 2-dimensi yang sering salah mengartikan kemiringan (slope) sebagai area terlarang. Pendekatan baru ini didasarkan pada peta biaya multi-lapis yang secara aktif mengintegrasikan informasi kemiringan selama tahap pembuatan peta lingkungan. Sebuah algoritma deteksi kemiringan dikembangkan untuk mengidentifikasi kemiringan dan secara adaptif mengubah peta biaya selama navigasi, sehingga kemiringan dianggap sebagai jalur yang dapat dilalui, hanya dengan biaya tambahan. Selain itu, untuk skenario di mana kemiringan mengarah ke lantai baru, sistem mengadopsi kode Aruco untuk memicu peralihan informasi peta dan sistem koordinat referensi yang sesuai, memungkinkan navigasi berlanjut tanpa gangguan.

Penelitian oleh (Palacín, Rubies, et al., 2023) membahas metode perencanaan jalur untuk robot pengirim paket yang bisa bekerja pada bangunan bertingkat. Paper ini menggunakan *static navigation tree* yang menggambarkan *weighted paths* dalam bangunan bertingkat. Lalu, *navigation tree* digunakan untuk merencanakan jalur dan mengestimasi durasi dari semua rute pengiriman menggunakan algoritma dijkstra.

Penelitian oleh (Zhu et al., 2020) mengusulkan metode *real-time* untuk mendeteksi dan lokalisasi tangga. Sensor LiDAR VLP-16 digunakan untuk mendeteksi fitur geometris dari tangga berupa bidang miring dan garis paralel. Sensor IMU 3-axis juga digunakan saat robot memanjat tangga dengan basis antara coupling antara roll dan yaw pada bidang miring.

Studi oleh (Palacín, Bitriá, et al., 2023) menyajikan prosedur bagi robot otonom untuk bernavigasi antar lantai di gedung bertingkat dengan menggunakan lift, dengan lokalisasi yang didasarkan pada algoritma Iterative Closest Point (ICP). Performa ICP dievaluasi dalam berbagai kondisi, yaitu saat pintu lift dalam proses terbuka dan tertutup.

Studi oleh (T. Kim et al., 2024) menguraikan pengembangan *indoor delivery mobile robot* dengan arsitektur yang dirancang khusus untuk lingkungan multi-lantai. Untuk mengatasi tantangan navigasi di gedung bertingkat, robot ini menggunakan peta navigasi terintegrasi yang dibuat dengan menggabungkan peta *point cloud* multi-lantai dan peta topologi berbasis node

graph, memungkinkan lokalisasi yang efektif dan perencanaan rute 3D yang mencakup pergerakan antar-lantai.

Penelitian oleh (Lee et al., 2023) memperkenalkan sistem kontrol navigasi lantai otonom untuk robot bergerak yang memanfaatkan pengenalan tombol lift dengan *deep learning* untuk memungkinkan robot beroperasi secara mandiri di antara lantai tanpa memerlukan modifikasi fasilitas lift yang ada atau sistem manajemen gedung tambahan.

Studi oleh (Xie & Zhou, 2023) membuat sistem navigasi yang dirancang khusus untuk melintasi lantai dan tangga dengan aman di lingkungan buatan manusia. Sistem ini mengintegrasikan algoritma A* untuk perencanaan jalur awal dan *DWA-based path following* untuk permukaan datar.

Paper oleh (E.-H. Kim et al., 2020) membahas navigasi multi-lantai robot layanan seluler, khususnya dalam hal masuk dan keluar lift. Penulis mengusulkan metode deteksi dan pengenalan fitur lift serta navigasi robot menggunakan Deep learning.

Paper oleh (Li et al., 2021) mengusulkan kerangka kerja Autonomous Elevator Button Recognition and Operation (AEBRO) yang memungkinkan robot melakukan navigasi gedung bertingkat dengan mengoperasikan lift secara otonom.

Paper oleh (Jung et al., 2024) mengusulkan skema untuk perencanaan lintasan menggunakan algoritma A* yang ditambah dengan fungsi biaya untuk mempertimbangkan faktor-faktor realistis seperti waktu tunggu lift, dan ini memungkinkan robot memilih mode pergerakan antar lantai (lift atau tangga) secara efisien berdasarkan titik awal, titik akhir, dan waktu tunggu lift.

2.2 Teori/Konsep Dasar

2.2.1 Layered Costmap

Costmap adalah representasi grid dua dimensi dari lingkungan robot yang digunakan dalam sistem navigasi, dengan tiap sel pada grid memiliki harga yang terasosiasi dengannya. Pada *costmap* modern, tiap sel merupakan integer dari 0 hingga 255 yang merepresentasikan biaya navigasi. Secara umum, algoritma perencanaan jalur akan mengutamakan melewati sel dengan harga rendah dan menghindari sel dengan harga tinggi. Metode *costmap* memberikan keseimbangan antara kualitas informasi dan efisiensi algoritma. Sebagai ekstensi dari *costmap*, digunakan konsep *layered costmap*, yaitu struktur costmap yang terbagi menjadi beberapa layer terpisah berdasarkan jenis data atau fungsi spesifik (misalnya static map, obstacle, inflation, prox-emic). Setiap layer memodifikasi master costmap secara terstruktur dan independen, memungkinkan sistem navigasi mempertimbangkan konteks yang lebih kompleks dan menghasilkan perilaku pergerakan robot yang lebih cerdas dan adaptif terhadap lingkungan (Macenski et al., 2023).

2.2.2 Lokalisasi Iterative Closest Point (ICP)

Lokalisasi robot adalah proses untuk menentukan posisi dan orientasi robot (pose) relatif terhadap peta lingkungan tempat robot beroperasi. Kemampuan ini sangat penting bagi robot untuk melakukan navigasi dan pengambilan keputusan yang tepat dalam menjalankan misi otonom. Lokalisasi umumnya menggunakan informasi dari sensor internal, seperti encoder roda dan IMU, dan sensor eksternal, seperti kamera atau LiDAR, untuk memperkirakan posisi robot berdasarkan model gerak dan observasi lingkungan.

Salah satu kategori metode lokalisasi adalah map-matching, dimana data sensor dari lingkungan dicoba untuk disamakan dengan fixed map yang sudah ada. Algoritma iterative closest point (ICP) merupakan salah satu algoritma yang bekerja dengan metode map-matching ini. Cara kerja algoritma ICP adalah dengan mencoba mengurangi jarak Euclidean antara input data dengan sebuah model referensi (dalam lokalisasi adalah fixed map). Secara matematis, bayangkan terdapat dua set titik 2D A dan B. Tujuan algoritma adalah menemukan sebuah transformasi A ke B untuk mencari mean squared distance (MSD) antara titik A dan B (Sobreira et al., 2019).

$$\text{MSD}(A, B, u) = \frac{1}{n} \sum_{a \in A, b \in B} \|b - u(a)\|^2 \quad (2.1)$$

Dengan rumus matematika diatas, algoritma ICP mencoba untuk mengurangi MSD pada setiap iterasi. ICP terbagi menjadi dua tahap, yaitu matching stage dan transformation stage. Matching stage adalah tahap pertama dari ICP dan bertujuan untuk meminimalkan nilai mean square distance dengan dengan matching terbaik antara set titik A dan B.

Transformation stage adalah tahap kedua yang bertujuan untuk mencari nilai paling optimal dari transformasi rotasi R dan translasi t untuk mencapai MSD paling kecil. ICP menggunakan simple least square solver untuk menemukan transformasi matrix (R — t) yang meminimalkan MSD. untuk mencapai hal tersebut dilakukan pengurangan dari data referensi dan point cloud sensor nilai centroid masing-masing, yang ditunjukkan di rumus dibawah (Sobreira et al., 2019).

$$a'_i = a_i - \frac{1}{n} \sum_{i=0}^n a_i \quad (2.2)$$

$$b'_i = b_i - \frac{1}{m} \sum_{i=0}^m b_i \quad (2.3)$$

Setelah itu, nilai kovarian silang matrix dikomputasi menggunakan persamaan dibawah dengan A' dan B' sama dengan set poin a'_i dan b'_i.

$$H = A' B'^T \quad (2.4)$$

Lalu, sudut rotasi θ bisa dihitung dengan rumus dibawah .

$$\theta = \text{atan2}((H(0, 1) - H(1, 0)), (H(0, 0) + H(1, 1))) \quad (2.5)$$

Pada akhir tahap transformasi, data sensor diubah menggunakan matriks (R — t) yang diperkirakan dan algoritma kembali ke tahap pencocokan, kecuali jika kriteria penghentian diverifikasi (misalnya, jumlah iterasi, peningkatan kesalahan Euclidean antar iterasi, dll.) (Sobreira et al., 2019).

$$R = m \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \quad (2.6)$$

$$t = \bar{b} - R\bar{a} \quad (2.7)$$

2.2.3 Path Planning A*

Path planning adalah proses perencanaan jalur geometris yang menghubungkan titik awal ke titik tujuan tanpa menabrak rintangan, biasanya dilakukan dalam ruang konfigurasi (configuration space) dengan mempertimbangkan berbagai batasan lingkungan. Metode *path planning* umum mencakup teknik roadmap, cell decomposition, dan artificial potential field.

Salah satu algoritma *path planning* yang paling sering digunakan adalah A*. A* dikategorikan sebagai algoritma best first search (BFS) yang menggabungkan kelebihan uniform-cost dan greedy searches dengan menggunakan fitness function.

$$f(n) = g(n) + h(n) \quad (2.8)$$

Pada persamaan diatas, $g(n)$ adalah biaya kumulatif dari titik awal ke simpul n , dan $h(n)$ adalah estimasi heuristik biaya tersisa menuju simpul tujuan. Selama pencarian, A* mengelola daftar terbuka (simpul yang akan dipertimbangkan) dan daftar tertutup (simpul yang sudah dikunjungi). Algoritma ini bekerja dengan memperluas simpul dari daftar terbuka dengan fungsi kecocokan minimal, memindahkannya ke daftar tertutup, dan menambahkan tetangganya ke daftar terbuka hingga simpul tujuan diperluas.

Pilihan heuristik yang baik sangat penting untuk kualitas dan efisiensi pencarian A*. Heuristik yang mengestimasi biaya nyata menjamin jalur terpendek, tetapi bisa memperluas terlalu banyak simpul. Sebaliknya, heuristik yang melebih-lebihkan dapat mempercepat pencarian ke tujuan, namun berisiko melambatkan jika jalur optimal menyimpang. Meskipun A* banyak digunakan karena menemukan jalur biaya minimum dan memastikan keberadaan jalur bebas, kelemahan utamanya adalah konsumsi waktu yang tinggi dan berat komputasi yang signifikan (Damle & Susan, 2022).

2.2.4 Path Tracking Pure Pursuit

Path tracking atau trajectory planning bertugas menambahkan informasi waktu pada jalur tersebut, sehingga menghasilkan lintasan yang dapat diikuti oleh robot secara fisik. Proses ini mempertimbangkan batasan kinematik dan dinamik dari robot, termasuk kecepatan, percepatan, dan jerk, untuk menghasilkan lintasan yang halus, aman, dan efisien. *path tracking* memastikan bahwa gerakan aktual robot mengikuti jalur yang telah direncanakan dengan akurat dan memastikan kestabilan robot serta performa yang sesuai dengan sistem kontrol yang digunakan (Yao et al., 2020).

Pure-pursuit adalah metode *path tracking* geometris yang efektif untuk robot otonom. Cara kerjanya adalah membayangkan kendaraan mengikuti sebuah titik di jalur yang diberikan, beberapa jarak di depannya. Titik ini dipilih pada jarak pandang ke depan (look-ahead distance) yang ditentukan. Implementasinya meliputi penentuan lokasi kendaraan saat ini, menemukan titik terdekat di jalur, memilih titik tujuan pada jarak pandang ke depan konstan, mengubah koordinat titik tujuan ke kendaraan, menghitung kelengkungan, dan memperoleh sudut kemudi. Keunggulan metode ini adalah kemudahan penyetelan jarak pandang ke depan, kesederhanaan komputasi, dan ketiadaan istilah turunan (Yao et al., 2020).

$$\text{Error}_{\text{cte}} = \text{Error}_{\text{calculate}} + \text{Error}_{\text{tracking}} \quad (2.9)$$

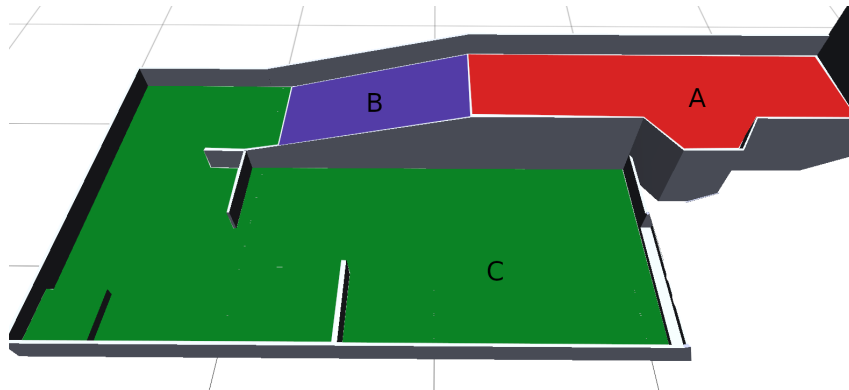
BAB III

METODOLOGI

3.1 Pengolahan Data Map

Sistem navigasi ini dirancang untuk beroperasi pada area yang petanya sudah diketahui sebelumnya (*known map*). Informasi lingkungan tersebut disimpan dalam bentuk *occupancy grid*. Tantangan utama yang dihadapi adalah bahwa data lingkungan yang dikumpulkan bersifat tiga dimensi (3D), sementara *occupancy grid* yang digunakan hanya memiliki dua dimensi (2D). Untuk menyetarakan perbedaan ini, lingkungan 3D dibagi menjadi beberapa *region* yang lebih kecil, dan setiap *region* direpresentasikan secara terpisah dalam bentuk peta 2D. Sehingga, setiap *region* menjadi proyeksi 2D dari sub-bagian lingkungan 3D. Pembagian *region* dalam ruang 3D dapat dilihat pada Gambar 3.1.

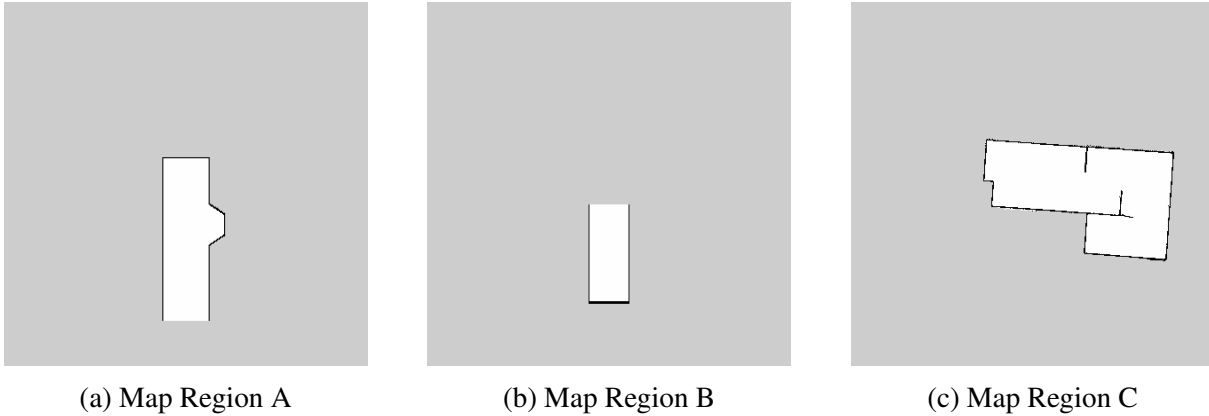
Selain peta utama, sistem navigasi juga membutuhkan peta transisi yang berfungsi sebagai jembatan penghubung antar *region*. Peta transisi adalah peta terpisah yang menyimpan informasi relasi spasial setiap *region* terhadap *global frame of reference* yang seragam, sehingga posisi absolut masing-masing *region* dalam koordinat global dapat diketahui. Dengan adanya peta transisi ini, robot dapat melakukan perpindahan antar *region* secara mulus selama navigasi, meskipun kedua *region* terpisah oleh area *unknown*.



Gambar 3.1: Lingkungan arena dalam 3D

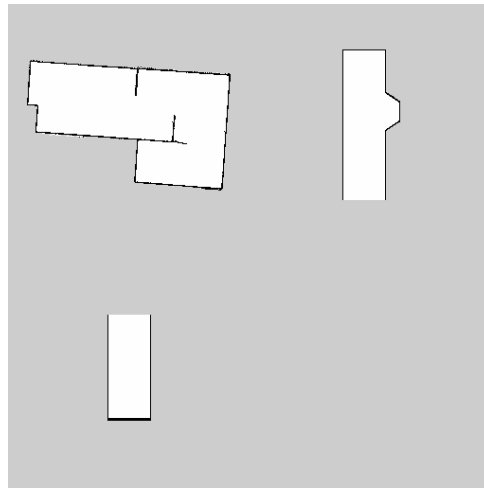
3.1.1 Map Region

Peta untuk setiap *region* diperoleh melalui proses *mapping* menggunakan Hector SLAM atau dibuat secara manual berdasarkan dimensi ukuran lingkungan asli. Hasil akhir peta masing-masing *region* ditunjukkan pada Gambar 3.2.



Gambar 3.2: Map Occupancy Grid masing-masing region

Sistem navigasi tidak menggunakan peta individu dari masing-masing *region* secara terpisah, melainkan menggunakan satu peta utama yang merupakan hasil kombinasi seluruh *region*. Dalam peta utama tersebut, setiap *region* dipisahkan oleh area *unknown* (wilayah yang tidak diketahui).



Gambar 3.3: Map Gabungan

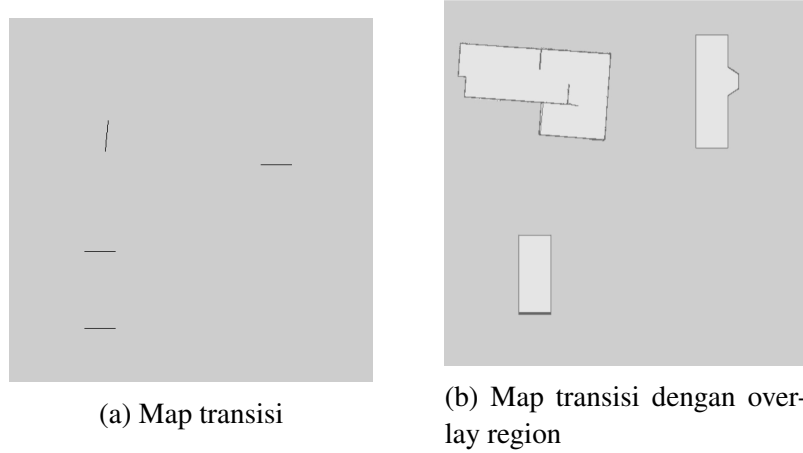
Untuk menghasilkan map gabungan, diperlukan deteksi Region of Interest (ROI) pada setiap peta pisah. ROI didefinisikan sebagai area yang mengandung piksel gelap (nilai $\geq$ threshold), yang mewakili obstacle atau peta yang bermakna. Untuk setiap peta, ROI diidentifikasi melalui:

$$\text{ROI} = \{(x, y) : I(x, y) < T\}$$

dimana, $I(x, y)$ adalah nilai intensitas piksel pada koordinat (x, y) , T adalah threshold

Peta gabungan akhir tersusun dari beberapa bagian, dan setiap bagian ditempati oleh satu *region*. Banyaknya bagian tersebut ditentukan oleh jumlah peta *region* yang dimasukkan ke dalam proses penggabungan.

3.1.2 Map Transisi



Gambar 3.4: Map transisi antar region

Peta transisi digunakan untuk menandakan perbatasan antar *region* dalam peta gabungan. Peta ini dibuat melalui proses anotasi terhadap peta gabungan, dimana pengguna secara manual menandai piksel-piksel yang mewakili perbatasan antar *region*. Proses anotasi ini menghasilkan 'metadata' yang menyimpan informasi sebagai berikut: (1) pasangan *region* yang terhubung, (2) orientasi setiap *region* terhadap *global frame of reference* dalam bentuk *quaternion*, serta (3) posisi inisial robot pada setiap *region* dalam koordinat piksel.

Tabel 3.1: Daftar Koneksi pasangan

Pasangan	Region	Pixel A (x, y)	Pixel B (x, y)
0	A – B	(354, 206)	(106, 328)
1	B – C	(106, 436)	(135, 187)

Koneksi antar *region* bersifat linear, artinya setiap *wormhole* hanya menghubungkan dua *region* yang berdekatan secara langsung. Sebagai contoh, *region* A hanya dapat terhubung ke *region* B, dan *region* B hanya dapat terhubung ke *region* C. Tidak ada koneksi langsung dari *region* A ke *region* C. Setiap pasangan direpresentasikan oleh dua piksel, yaitu *pixel A* dan *pixel B*, yang menandai lokasi garis transisi pada peta transisi.

Tabel 3.2: Orientasi Region terhadap Global Frame

Region	Quaternion			
	x	y	z	w
A	0.0	0.0	0.0	1.0
B	0.0	0.0	0.0	1.0
C	0.0	0.0	-0.7046	0.7096

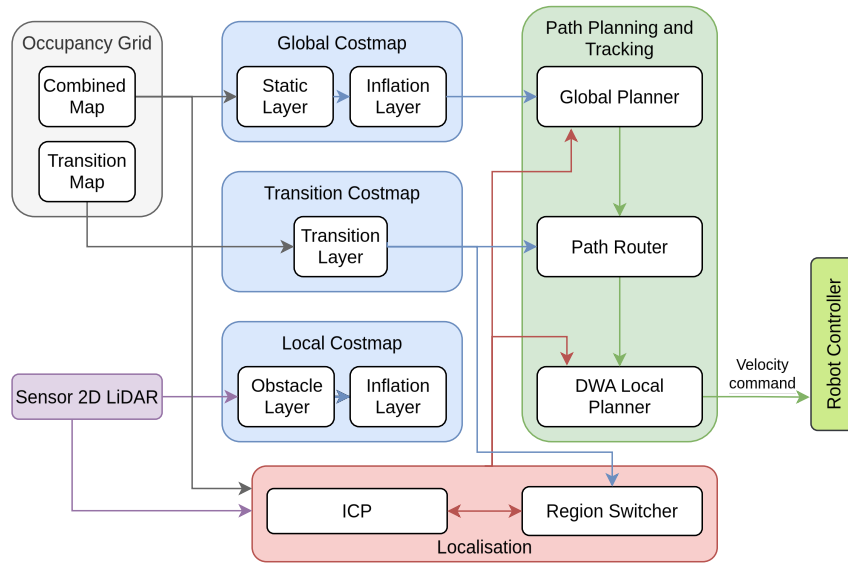
Orientasi digunakan untuk menyelaraskan arah peta setiap *region* dengan arah yang seragam dalam *global frame of reference*. Dalam kasus ini, *region* A dan B memiliki orientasi yang sama, sedangkan *region* C memiliki rotasi berbeda dengan A dan B.

Tabel 3.3: Posisi Inisial Robot

Parameter	Nilai pixel
Posisi inisial	(375, 75)

Posisi awal robot ditentukan secara manual dalam koordinat piksel, lalu dikonversi ke koordinat dunia menggunakan informasi orientasi dan resolusi peta. Posisi ini digunakan sebagai titik awal untuk dalam sistem navigasi.

3.2 Perancangan Sistem Navigasi



Gambar 3.5: Arsitektur sistem navigasi

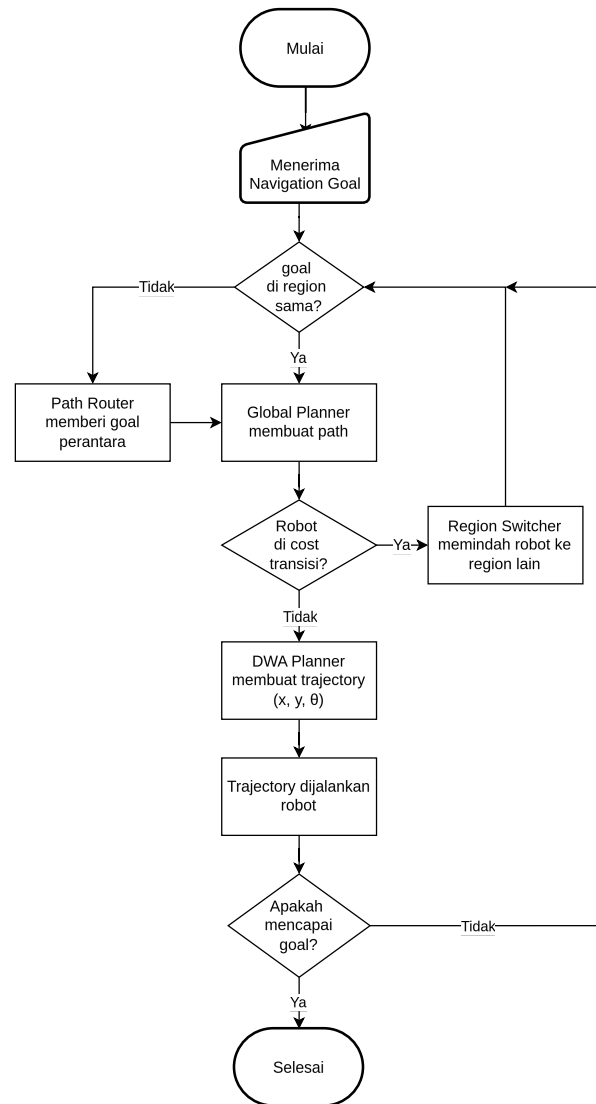
Sistem navigasi ini berfungsi untuk merencanakan dan mengeksekusi pergerakan robot dari satu region ke region lain pada lingkungan yang telah diketahui. Sistem dapat dibagi menjadi empat komponen utama, yaitu occupancy grid, costmap, lokalisasi, serta path planning dan tracking. Occupancy grid menyediakan representasi diskrit dari lingkungan. Costmap menyimpan informasi tersebut dalam bentuk peta biaya (costmap). Lokalisasi bertugas melacak posisi dan region robot secara real-time. Path planning dan tracking bertanggung jawab menghasilkan jalur antar region dan menjalankannya.

Data combined map (peta gabungan) digunakan untuk membangun global costmap, dimulai dari pembentukan static layer yang merepresentasikan obstacle yang diketahui dari peta. Selain itu, combined map juga menjadi basis bagi lokalisasi ICP untuk dikomparasi dengan hasil scan LiDAR guna memperkirakan pose robot. Sementara itu, map transisi digunakan sebagai sumber data bagi transition costmap, yang menyimpan informasi koneksi antar region.

Costmap dalam sistem ini dibagi menjadi tiga lapisan, yaitu global costmap, local costmap, dan transition costmap. Global dan local costmap sama-sama menggunakan inflation layer, namun dengan sumber data yang berbeda. Inflation layer pada global costmap dibangun berdasarkan static layer yang berasal dari obstacle tetap pada peta. Inflasi pada local costmap didasarkan pada obstacle layer yang diperbarui secara dinamis dari data scan LiDAR.

Global costmap digunakan oleh global planner untuk merencanakan path utama menuju tujuan. Untuk navigasi antar region, path router bertugas merutekan ulang jalur menuju zona transisi ketika tujuan berada di region yang berbeda. Hasil rutekan tersebut kemudian diteruskan ke DWA local planner, yang menghasilkan trajectory berupa kecepatan x , y , dan rotasi yang diberikan ke kontroler robot untuk dieksekusi.

Untuk memfasilitasi tracking, lokalisasi ICP dan region switcher bekerja sama melacak posisi koordinat robot serta region tempat robot berada saat ini. Ketika robot mencapai zona transisi, region switcher melakukan perpindahan region dan memperbarui posisi awal ICP pada region yang baru.



Gambar 3.6: Diagram alur navigasi

3.2.1 Lokalisasi Iterative Closest Point (ICP)

Sistem navigasi ini menggunakan algoritma Iterative Closest Point (ICP) yang digunakan untuk memperkirakan koreksi pose robot terhadap static occupancy grid map. ICP bekerja dengan mencocokkan titik-titik dari laser scan dengan titik-titik yang sesuai pada peta, lalu menghitung transformasi optimal yang meminimalkan jarak antara pasangan titik tersebut. Proses ini iteratif, di mana setiap iterasi memperbaiki estimasi pose robot hingga konvergen ke solusi yang stabil. Sebelum ICP, dilakukan prekomputasi KNN untuk mempercepat pencarian pasangan titik selama proses ICP.

3.2.1.1 Prekomputasi KNN

Sistem ini melakukan prekomputasi atas semua kemungkinan pasangan titik (sensor point $\rightarrow$ map point) saat menerima map baru, lalu menyimpan informasi tersebut di dalam hash table. Pendekatan ini dimungkinkan karena occupancy grid bersifat diskrit dan diketahui secara penuh. Setiap sel bebas di dekat tembok dapat diketahui sebelumnya K buah sel rintangan terdekatnya. Proses ini disebut interpretasi map dan terjadi sekali saat startup, bukan saat runtime.

Hasilnya lookup KNN saat runtime menjadi $O(1)$.

Untuk setiap sel rintangan (occupied) pada indeks i_{occ} :

1. Cek kejangkauan menggunakan Moore neighbourhood 3×3 (8 tetangga terdekat). Jika tidak ada satupun sel bebas di antara tetangganya, maka sel rintangan tersebut berada di dalam tembok (interior) dan tidak akan pernah terdeteksi oleh laser hanya mengenai permukaan tembok, bukan interiornya. Sel seperti ini diabaikan untuk menghemat memori dan komputasi.
2. Iterasi area sekitar sel rintangan yang dapat dicapai. Semakin besar r , semakin banyak sel yang akan memiliki KNN precomputed, tetapi juga semakin besar ukuran hash table.
3. Untuk setiap sel tetangga pada indeks i_{nb} yang berada dalam radius r , *world coordinates* dari sel rintangan i_{occ} dimasukkan ke dalam daftar KNN milik i_{nb} . Daftar ini dijaga tetap terurut berdasarkan jarak Euclidean dan dibatasi maksimal K entry. Jika sudah penuh dan jarak lebih besar dari entry terjauh, data diabaikan, namun jika jarak lebih kecil, entry terjauh dihapus dan data baru dimasukkan pada posisi yang tepat.

3.2.1.2 Proses Data LiDAR

Data scan dari sensor LiDAR diproses terlebih dahulu dan disimpan dalam format seperti berikut. Untuk scan dengan sinar N pada sudut $\theta_0, \theta_1, \dots, \theta_{N-1}$:

$$\mathbf{C} = \begin{bmatrix} \cos \theta_0 & \cos \theta_1 & \cdots & \cos \theta_{N-1} \\ \sin \theta_0 & \sin \theta_1 & \cdots & \sin \theta_{N-1} \end{bmatrix} \quad (3.1)$$

Cache hanya dihitung ulang ketika parameter scan (jumlah sinar, rentang sudut, increment) berubah.

Diberikan pembacaan vektor $r_0, r_1, \dots, r_{N-1}$:

$$\mathbf{S} = \begin{bmatrix} r_0 \cos \theta_0 & r_1 \cos \theta_1 & \cdots & r_{N-1} \cos \theta_{N-1} \\ r_0 \sin \theta_0 & r_1 \sin \theta_1 & \cdots & r_{N-1} \sin \theta_{N-1} \end{bmatrix} \quad (3.2)$$

$\mathbf{S}$ adalah hasil *scan point cloud* dalam koordinat cartesian, hasil konversi dari data laser mentah ke titik-titik 2D.

3.2.1.3 Loop Utama

Algorithm 1 ICP Single Iteration

Require: *scan* ▷ N points dalam sensor frame
Ensure: *new_to_old* ▷ transform_t

```

1:  $T \leftarrow \text{Identity}$ 
2: for  $\text{iteration} = 1$  to  $\text{max\_iter}$  do
3:    $\text{random\_scan} \leftarrow \text{sampler}(\text{scan})$  ▷ subsample dengan random stride
4:    $\text{data} \leftarrow T \cdot \text{random\_scan}$  ▷ transform ke map frame
5:    $M \leftarrow \text{matches}(\text{data}, \text{knn})$  ▷ cari correspondences
6:    $w \leftarrow \text{get\_weights}(M)$  ▷ weight setiap match
7:    $T\_update \leftarrow \text{point\_to\_map}(M, w)$  ▷ SVD solver
8:    $T \leftarrow T\_update \cdot T$  ▷ akumulasi
9:   if  $\text{converged}(T\_update)$  then
10:    break
11:   end if
12: end for
13: return  $T$ 

```

3.2.1.4 Correspondence Matching

Correspondence matching adalah langkah yang memasangkan setiap titik dari laser scan dengan titik terdekat dari map. Proses ini dilakukan dengan memanfaatkan hash table KNN yang sudah diprekomputasi sebelumnya. Hasil akhirnya digunakan untuk menghitung transformasi optimal melalui SVD solver.

Untuk setiap sensor point s_j :

1. Cek apakah s_j berada di dalam map bounds. Jika tidak, lewati s_j .
2. melakukan lookup ke hash table untuk mendapatkan daftar hingga K titik map terdekat yang mungkin menjadi pasangan
3. Duplikasi s_j untuk setiap nearest neighbors-nya, menghasilkan $(s_j, m_{j,1}), (s_j, m_{j,2}), \dots, (s_j, m_{j,K})$. Ini memperluas N sensor points menjadi hingga $N \times K$ pasangan yang cocok.

3.2.1.5 Solusi SVD

Solusi SVD ini digunakan untuk menghitung transformasi yang memetakan titik-titik dari laser scan ke titik-titik yang sesuai di map, berdasarkan pasangan yang ditemukan pada langkah correspondence matching.

Diberikan M pasangan $\{s_i\}$ (sensor) dan $\{m_i\}$ (map) dengan weights $\{w_i\}$, kita mencari rigid transform $T \in SE(2)$ yang meminimalkan transformasi. $SE(2)$ berarti transformasi yang mempertahankan jarak dan orientasi.

$$T^* = \arg \min_{R, t} \sum_{i=1}^M w_i \|m_i - (Rs_i + t)\|^2 \quad (3.3)$$

Langkah 1: Kalkulasi *Weighted Centroids*

$$\bar{s} = \frac{\sum_{i=1}^M w_i s_i}{\sum_{i=1}^M w_i}, \quad \bar{m} = \frac{\sum_{i=1}^M w_i m_i}{\sum_{i=1}^M w_i} \quad (3.4)$$

Langkah 2: Centering (pindahkan titik ke pusat koordinat)

$$s'_i = s_i - \bar{s}, \quad m'_i = m_i - \bar{m} \quad (3.5)$$

Langkah 3: Cross-Covariance Matrix

$$H = \sum_{i=1}^M w_i m'_i (s'_i)^\top = M W S^\top \quad (3.6)$$

dimana $S, M \in \mathbb{R}^{2 \times M}$ adalah matriks yang kolomnya berisi titik-titik yang sudah di-center, dan $W = \text{diag}(w_1, \dots, w_M)$.

Langkah 4: Singular Value Decomposition

$$H = U \Sigma V^\top \quad (3.7)$$

dengan $U, V \in \mathbb{R}^{2 \times 2}$ orthonormal dan $\Sigma = \text{diag}(\sigma_1, \sigma_2)$.

Langkah 5: Rotasi Optimal

$$R = UV^\top \quad (3.8)$$

Jika $\det(R) < 0$ (menghasilkan refleksi, bukan rotasi murni), koreksi dengan membalik tanda baris pertama $V^\top$:

$$\text{if } \det(R) < 0 : \quad V_{0,:}^\top \leftarrow -V_{0,:}^\top, \quad R = UV^\top \quad (3.9)$$

Langkah 6: Translasi Optimal

$$t = \bar{m} - R\bar{s} \quad (3.10)$$

Langkah 7: Susun Rigid Transform 2D

$$T_{\text{update}} = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} \cos \Delta\theta & -\sin \Delta\theta & t_x \\ \sin \Delta\theta & \cos \Delta\theta & t_y \\ 0 & 0 & 1 \end{bmatrix} \quad (3.11)$$

Setelah iterasi, cek apakah transform update cukup kecil:

$$\text{converged} \iff \|t\| < t_{\text{norm}} \wedge \left| \arctan 2(R_{1,0}, R_{0,0}) \right| < r_{\text{norm}} \quad (3.12)$$

Untuk mempercepat, hanya 1/s dari scan points yang digunakan:

1. Pilih offset acak $o \in [0, s - 1]$
2. Ambil setiap point ke- s mulai dari o :

$$p'_k = p_{o+k \cdot s}, \quad k = 0, 1, \dots, \left\lfloor \frac{N}{s} \right\rfloor - 1 \quad (3.13)$$

Tabel 3.4: Parameter Default ICP

Parameter	Nilai Default	Keterangan
max_iter	50	Iterasi maksimum ICP
s	2	<i>Random stride</i> subsampling
t_{norm}	0.01 m	Batas konvergensi translasi
r_{norm}	0.01 rad ($\approx 0.57^\circ$)	Batas konvergensi rotasi
k	8	Jumlah <i>nearest neighbors</i>
r	10	Radius pencarian <i>neighbor</i>

3.2.2 Costmap

Costmap adalah representasi grid yang menyimpan informasi biaya untuk navigasi. Setiap sel dalam costmap memiliki nilai biaya yang mencerminkan kesulitan atau risiko melewati area tersebut. Nilai biaya ini dapat dipengaruhi oleh berbagai faktor seperti keberadaan rintangan. Costmap menyimpan informasi map grid 2D dalam bentuk array 1D

Tabel 3.5: Nilai konstanta costmap

Nama Konstanta	Nilai	Makna
NO_INFORMATION	255	Tidak diketahui (<i>unknown</i>)
LETHAL_OBSTACLE	254	Obstacle mematikan
INSCRIBED_INFLATED_OBSTACLE	253	Batas terluar inflasi obstacle
INFLATED_OBSTACLE	6-252	Inflasi obstacle
TRANSITION_CELL	1-5	Area transisi antar region
FREE_SPACE	0	Ruang bebas

Indeks Grid

konversi dari koordinat grid 2D (m_x, m_y) ke flat index 1D:

$$i = m_y \cdot W + m_x \quad (3.14)$$

dimana W adalah lebar grid (jumlah cell per baris).

Koordinat Dunia - Grid

Diberikan origin grid di (o_x, o_y) dan resolusi r (meter per cell):

World $\rightarrow$ Grid:

$$m_x = \left\lfloor \frac{w_x - o_x}{r} \right\rfloor, \quad m_y = \left\lfloor \frac{w_y - o_y}{r} \right\rfloor \quad (3.15)$$

Grid $\rightarrow$ World :

$$w_x = o_x + \left(m_x + \frac{1}{2}\right) \cdot r, \quad w_y = o_y + \left(m_y + \frac{1}{2}\right) \cdot r \quad (3.16)$$

1.4 Rolling Window

Untuk local costmap yang bergerak mengikuti robot, origin grid digeser:

$$\Delta_x = \lfloor \text{new_origin}_x - \text{old_origin}_x \rfloor, \quad \Delta_y = \lfloor \text{new_origin}_y - \text{old_origin}_y \rfloor \quad (3.17)$$

Array costmap digeser sebesar (Δ_x, Δ_y) dan cell baru diisi dengan NO_INFORMATION.

3.2.2.1 Static Layer

Static layer adalah lapisan costmap paling dasar yang menyimpan informasi *known obstacle* atau tembok yang sudah diketahui. Lapisan costmap ini mengkonversi setiap sel dari nilai occupancy grid (0-100) ke nilai biaya costmap internal (0-255). Sel yang tidak diketahui (unknown) diberi biaya tertinggi, sel bebas diberi biaya terendah, dan sel yang terisi (occupied) diberi biaya tinggi.

$$C_{\text{internal}} = \begin{cases} 255 & \text{if } OG = -1 \text{ (unknown)} \\ 0 & \text{if } OG = 0 \text{ (free)} \\ 254 & \text{if } OG \geq 100 \text{ (occupied)} \end{cases} \quad (3.18)$$

3.2.2.2 Obstacle Layer

Obstacle layer dirancang untuk menangani objek-objek dinamis atau rintangan yang baru terdeteksi di lingkungan. Informasi pada lapisan ini diperoleh secara real-time dari sensor LiDAR robot, yang terus-menerus memindai area sekitar dan memperbarui *costmap* sesuai dengan obstacle yang terdeteksi. Lapisan ini memiliki batasan jangkauan observasi maksimum, yang jaraknya disesuaikan berdasarkan jarak *pointcloud* maksimum dari sensor LiDAR, yang berarti hanya rintangan dalam radius pandang sensor yang akan direpresentasikan di lapisan *costmap* ini.

Untuk setiap beam laser dengan range r dan sudut θ :

1. Hitung endpoint dalam frame sensor: $(r \cos \theta, r \sin \theta)$
2. Transform ke frame map via TF
3. Konversi ke cell: $(m_x, m_y) = \text{worldToMap}(x, y)$
4. Set cell ke LETHAL_OBSTACLE (254)

Gunakan algoritma Bresenham 2D untuk menggambar garis dari robot ke endpoint sensor. Semua cell yang dilewati garis (sebelum endpoint) di-clear:

$$C_{\text{master}}(m_x, m_y) = 0 \quad \forall \text{cell di sepanjang ray (kecuali endpoint)} \quad (3.19)$$

Bresenham 2D Raytracing

Algoritma Bresenham memilih pixel-pixel yang mendekati garis lurus antara dua titik grid. Untuk setiap langkah, algoritma memutuskan apakah akan bergerak di sumbu x, y, atau keduanya berdasarkan error accumulator:

Diberikan titik awal (x_0, y_0) dan titik akhir (x_1, y_1) :

$$\Delta_x = |x_1 - x_0|, \quad \Delta_y = |y_1 - y_0| \quad (3.20)$$

$$\text{err} = \Delta_x - \Delta_y, \quad s_x = \text{sign}(x_1 - x_0), \quad s_y = \text{sign}(y_1 - y_0) \quad (3.21)$$

Algorithm 2 Bresenham Raytracing

```

1: while  $x \neq x_1 \vee y \neq y_1$  do
2:    $e_2 \leftarrow 2 \cdot \text{err}$ 
3:   if  $e_2 > -\Delta_y$  then
4:      $\text{err} \leftarrow \text{err} - \Delta_y$ 
5:      $x \leftarrow x + s_x$ 
6:   end if
7:   if  $e_2 < \Delta_x$  then
8:      $\text{err} \leftarrow \text{err} + \Delta_x$ 
9:      $y \leftarrow y + s_y$ 
10:  end if
11:  raytrace_function( $x, y$ )
12: end while

```

3.2.2.3 Inflation Layer

Inflation layer berperan penting dalam memastikan pergerakan robot dapat menghindari berhimpitan dengan rintangan. Lapisan ini menciptakan zona inflasi di sekitar setiap rintangan yang terdeteksi dan memiliki nilai harga navigasi yang tinggi. Radius zona inflasi ini variabel yang ditentukan berdasarkan dimensi fisik atau *footprint* robot sehingga mencegah robot terlalu dekat dengan rintangan dan berpotensi menabrak. Lapisan ini secara efektif "memperbesar" rintangan di *costmap*, sehingga algoritma perencanaan jalur akan menjaga jarak aman dari objek-objek tersebut dari robot.

Lapisan inflasi berfungsi seperti safe distance minimum robot dari rintangan, namun dapat bekerja lebih dinamis karena memperhitungkan orientasi dan bentuk badan robot. Dengan bentuk robot yang unik, radius zona inflasi dapat berubah secara adaptif sesuai orientasi robot terhadap rintangan.

Layer ini memperluas obstacle dengan gradient cost, sehingga perencanaan path dapat menjaga jarak dari obstacle.

Algoritma Inflasi

Untuk setiap obstacle cell, terapkan cost ke semua cell dalam radius inflasi R .

Cost pada jarak d dari obstacle dihitung dengan fungsi eksponensial:

$$C(d) = \begin{cases} 253 \cdot e^{-\alpha \cdot (d - r_{\text{inscribed}})} & \text{if } d \leq R \\ 0 & \text{otherwise} \end{cases} \quad (3.22)$$

Atau dalam bentuk weight:

$$C(d) = 252 \cdot (e^{-\alpha \cdot (d - r_{\text{inscribed}})}) \quad (3.23)$$

dimana: d = jarak Euclidean dari cell ke obstacle terdekat, R = inflation radius (jarak maksimum inflasi), $r_{\text{inscribed}}$ = inscribed radius (radius robot, cost = 253 pada jarak ini), dan α = scaling factor (mengontrol kecuraman kurva cost). Nilai cost dibulatkan ke integer 6–252.

3.2.2.4 Transition Layer

Transition layer adalah lapisan yang menambahkan data area transisi ke dalam costmap. Data ini digunakan oleh untuk mendeteksi kapan robot mendekati batas transisi antar region dan melakukan "lompatan" pose antar area.

$$C_{\text{internal}} = \begin{cases} 255 & \text{if } OG = -1 \text{ (unknown)} \\ 0 & \text{if } OG = 0 \text{ (free)} \\ 1 & \text{if } OG \geq 100 \text{ (occupied)} \end{cases} \quad (3.24)$$

Lapisan ini juga memperluas area transisi dengan radius tertentu, sehingga robot dapat mendeteksi transisi sebelum benar-benar mencapai garis transisi. Hal ini membuat buffer zona deteksi di sekitar data transisi.

Tiap region di map memiliki metadata transisi unik yang menyimpan informasi relasi antar peta. Informasi tersebut termasuk orientasi relatif terhadap orientasi global yang dijadikan acuan.

3.2.3 Global Planner

Global planner menggunakan algoritma A* untuk menghitung potential field dari goal menuju start. Setiap sel dalam costmap diberi nilai potensial, yang semakin jauh dari goal, semakin tinggi nilainya. Path kemudian diekstrak dengan gradient descent dari start menuju goal, sebuah proses yang menurunkan nilai potensial.

3.2.3.1 Kalkulasi Cost

Kalkulasi cost di global planner adalah proses transformasi nilai costmap global (0–255) menjadi effective traversal cost yang digunakan dalam wavefront expansion (A*).

$$c_{\text{eff}} = \text{costmap}[n] \cdot f + c_{\text{neutral}} \quad (3.25)$$

$$c_{\text{eff}} = \begin{cases} c_{\text{eff}} & \text{if } c_{\text{eff}} < c_{\text{lethal}} \\ c_{\text{lethal}} - 1 & \text{if } c_{\text{eff}} \geq c_{\text{lethal}} \\ c_{\text{lethal}} & \text{if costmap}[n] \geq c_{\text{lethal}} \text{ dan tidak unknown} \end{cases} \quad (3.26)$$

f adalah cost factor dan c_{neutral} adalah neutral cost. Jika hasilnya melebihi lethal cost (253), nilai di batasi ke lethal cost - 1 (252) agar tidak sama persis dengan nilai lethal, dan jika nilai costmap mentah sudah mencapai lethal threshold maka c_{eff} langsung dikembalikan sebagai 253 yaitu obstacle yang tidak bisa dilalui.

Tabel 3.6: Parameter Default Cost Function

Parameter	Nilai Default	Keterangan
f	0.55	<i>Cost factor</i>
c_{neutral}	1	<i>Neutral cost</i>
c_{lethal}	253	<i>Lethal cost</i>

3.2.3.2 Kalkulasi Potensial Kuadratik

Kalkulasi potensial kuadratik adalah aproksimasi Euclidean wavefront yang lebih akurat. Diberikan dua neighbor terkecil dari arah horizontal dan vertikal:

$$t_c = \min(P_{\text{left}}, P_{\text{right}}), \quad t_a = \min(P_{\text{up}}, P_{\text{down}}) \quad (3.27)$$

Selisih: $\Delta = |t_c - t_a|$ (jika $t_c < t_a$, tukar).

Jika $\Delta \geq c_{\text{eff}}$:

$$P(n) = t_a + c_{\text{eff}} \quad (3.28)$$

Jika $\Delta < c_{\text{eff}}$ (interpolasi kuadratik):

$$d = \frac{\Delta}{c_{\text{eff}}} \quad (3.29)$$

$$v = -0.2301 \cdot d^2 + 0.5307 \cdot d + 0.7040 \quad (3.30)$$

$$P(n) = t_a + c_{\text{eff}} \cdot v \quad (3.31)$$

Koefisien polinomial ‘-0.2301, 0.5307, 0.7040’ adalah hasil least-squares fit dari fungsi $\sqrt{1 + d^2}$ (jarak Euclidean sebenarnya dari point source). Kurva ini mendekati:

$$v(d) \approx \sqrt{1 + d^2} \quad (3.32)$$

dengan error minimal.

3.2.3.3 A*

menggunakan heuristic untuk memprioritaskan ekspansi ke arah goal:

$$\text{priority} = P(n) + \text{ManhattanDistance}(n, \text{goal}) \cdot c_{\text{neutral}} \quad (3.33)$$

$$\text{ManhattanDistance}(x, y) = |x_{\text{goal}} - x| + |y_{\text{goal}} - y| \quad (3.34)$$

kalkulasi potential linear (Manhattan):

$$P(n) = \min(P_{\text{left}}, P_{\text{right}}, P_{\text{up}}, P_{\text{down}}) + c_{\text{eff}} \quad (3.35)$$

Heuristic ini membuat A* lebih cepat dari Dijkstra karena mengeksplorasi lebih sedikit cell, terutama ketika goal jauh. A* berekspansi seperti ellips yang condong ke start

3.2.3.4 Clear Endpoint

Setelah potential selesai, area 2x2 cell di sekitar goal di-clear:

Ini memastikan gradient path bisa menemukan jalan ke goal area meskipun goal berada di obstacle.

3.2.3.5 Ekstraksi Gradient Path

Gradient path bertujuan menghasilkan path halus dengan interpolasi sub-sel. dengan step = 0.5 sel per langkah.

Komputasi gradient sebagai berikut

Aproksimasi central difference:

$$g_x = \frac{P(x-1, y) - P(x+1, y)}{2} \quad (3.36)$$

$$g_y = \frac{P(x, y-1) - P(x, y+1)}{2} \quad (3.37)$$

Normalisasi:

$$\hat{g}_x = \frac{g_x}{\sqrt{g_x^2 + g_y^2}}, \quad \hat{g}_y = \frac{g_y}{\sqrt{g_x^2 + g_y^2}} \quad (3.38)$$

Untuk sel yang berada di obstacle (potensi tinggi), gradient dihitung dari tetangga terdekat yang valid.

Perhitungan Path Step:

$$\text{step} = \frac{\text{pathStep}}{\sqrt{g_x(p)^2 + g_y(p)^2}} \quad (3.39)$$

$$d_x \leftarrow d_x + g_x(p) \cdot \text{step}, \quad d_y \leftarrow d_y + g_y(p) \cdot \text{step} \quad (3.40)$$

Cell overflow: Ketika $|d_x| \geq 1.0$ atau $|d_y| \geq 1.0$, pindah ke cell tetangga:

3.2.3.6 Path Antar Region

Untuk memungkinkan global planner bekerja antar region, digunakan path interceptor yang memodifikasi planner untuk memungkinkan navigasi multi-region melalui transisi. Jika goal berada di area yang berbeda dari posisi robot saat ini, node ini menyusun plan bertahap melalui serangkaian poin transisi.

Daripada mengirim goal langsung yang mungkin tidak bisa dijangkau karena area berbeda secara topologi, path interceptor melakukan langkah-langkah berikut:

1. Menentukan rute area: region A $\rightarrow$ region B $\rightarrow$ region C $\rightarrow$...
2. Mengirim goal bertahap ke setiap transition point
3. Menunggu robot melompat (costmap jumper) sebelum melanjutkan ke step berikutnya
4. Mengirim final goal hanya setelah semua transisi selesai

Selama proses berjalan, path interceptor melakukan pelacakan posisi robot dan mengupdate region sekarang berdasarkan posisi robot dari ICP:

3.2.4 Local Planner DWA

Algoritma *Dynamic Window Approach* (DWA) bekerja dalam lima langkah utama. Pertama, algoritma mengambil sampel semua kemungkinan kecepatan (v_x, v_y, ω) yang dapat dicapai dalam satu siklus kendali berdasarkan *dynamic window*. Kedua, setiap sampel kecepatan disimulasikan menjadi sebuah *trajectory* menggunakan model kinematik robot. Ketiga, setiap *trajectory* diberi skor berdasarkan empat kriteria: jarak terhadap *global plan*, jarak terhadap *goal*, jarak terhadap *obstacle*, serta pencegahan osilasi. Keempat, *trajectory* dengan skor terbaik dipilih. Kelima, perintah kecepatan dari *trajectory* terbaik tersebut dikirim ke robot.

DWA Velocity window dibatasi oleh apa yang dapat dicapai dalam satu siklus kontrol:

$$v_{\min} = \max(v_{\min_limit}, v_{\text{current}} - \dot{v}_{\max} \cdot \Delta t) \quad (3.41)$$

$$v_{\max} = \min(v_{\max_limit}, v_{\text{current}} + \dot{v}_{\max} \cdot \Delta t) \quad (3.42)$$

dimana Δt adalah waktu satu siklus kontrol.

3.2.4.1 Velocity Sampling

Untuk setiap dimensi (v_x, v_y, ω), rentang dibagi menjadi sample merata:

$$\Delta v_x = \frac{v_{x,\max} - v_{x,\min}}{N_{v_x} - 1} \quad (3.43)$$

$$\Delta v_y = \frac{v_{y,\max} - v_{y,\min}}{N_{v_y} - 1} \quad (3.44)$$

$$\Delta\omega = \frac{\omega_{\max} - \omega_{\min}}{N_{\omega} - 1} \quad (3.45)$$

Total trajectory yang di-sample:

$$N_{\text{total}} = N_{v_x} \times N_{v_y} \times N_{\omega} \quad (3.46)$$

Konfigurasi dari yaml: $N_{v_x} = 6, N_{v_y} = 6, N_{\omega} = 30 \rightarrow$ total 1080 trajectories per evaluasi.

3.2.4.2 Model Kinematik

Untuk setiap sample kecepatan (v_x, v_y, ω) , trajectory disimulasikan menggunakan model kinematik diferensial:

$$x_{t+1} = x_t + \left(v_x \cos \theta_t + v_y \cos \left(\frac{\pi}{2} + \theta_t \right) \right) \cdot \Delta t \quad (3.47)$$

$$y_{t+1} = y_t + \left(v_x \sin \theta_t + v_y \sin \left(\frac{\pi}{2} + \theta_t \right) \right) \cdot \Delta t \quad (3.48)$$

$$\theta_{t+1} = \theta_t + \omega \cdot \Delta t \quad (3.49)$$

Jumlah langkah simulasi ditentukan oleh granularity:

$$N_{\text{steps}} = \frac{T_{\text{sim}}}{\Delta t} \quad (3.50)$$

$$\Delta t = \min \left(\frac{\text{sim_granularity}}{|v|}, \frac{\text{angular_sim_granularity}}{|\omega|} \right) \quad (3.51)$$

3.2.4.3 Penilaian Trayektori

$$C_{\text{total}} = C_{\text{osc}} + w_{\text{obs}} \cdot C_{\text{obs}} + w_{\text{gf}} \cdot C_{\text{gf}} + w_{\text{align}} \cdot C_{\text{align}} + w_{\text{path}} \cdot C_{\text{path}} + w_{\text{goal}} \cdot C_{\text{goal}} + w_{\text{twirl}} \cdot C_{\text{twirl}} \quad (3.52)$$

$$C_{\text{obs}} = \begin{cases} \sum_{k=1}^N C_k & \text{if sum_scores} \\ \max_k C_k & \text{otherwise} \end{cases} \quad (3.53)$$

Dilakukan juga Collision Checking. Setiap titik sepanjang trajectory, footprint robot di-rasterize ke costmap:

Menggunakan Bresenham line drawing untuk menelusuri setiap tepi footprint polygon di grid costmap.

Semua nilai di agregasi menjadi satu skor total untuk trajectory tersebut. Trajectory dengan skor terendah dipilih sebagai perintah kecepatan yang akan dieksekusi.

Mode sum scores:

$$C_{\text{obs}} = \begin{cases} \sum_{k=1}^N C_k & \text{if sum_scores} \\ \max_k C_k & \text{otherwise} \end{cases} \quad (3.54)$$

3.2.4.4 kalkulasi Cost

Tujuan menilai trajectory berdasarkan jarak ke global plan atau goal

Wavefront Propagation

““

$$d_{\text{cell}} = \text{number of BFS steps from nearest target cell} \quad (3.55)$$

Scoring Mode

Untuk setiap titik pada trajectory, nilai ‘target dist’ dari MapGrid dibaca.

Forward Point Shift

$$p_{\text{score}} = p_{\text{robot}} + d_{\text{forward}} \cdot \begin{pmatrix} \cos \theta \\ \sin \theta \end{pmatrix} \quad (3.56)$$

Path Costs (C_{path})

Weight: $w_{\text{path}} = \text{resolution} \cdot \text{path_distance_bias}$

Goal Costs (C_{goal})

Weight: $w_{\text{goal}} = \text{resolution} \cdot \text{goal_distance_bias}$

Goal Front Costs (C_{gf})

Menggunakan forward point shift. Menilai apakah **hidung robot** mengarah ke goal.

$$p_{\text{hidung}} = p_{\text{robot}} + d_{\text{forward}} \cdot \hat{v} \quad (3.57)$$

$$C_{\text{gf}} = \text{target_dist}(p_{\text{hidung}}) \quad (3.58)$$

Alignment Costs (C_{align})

Sama dengan path costs, tetapi dengan forward point shift. Menilai apakah hidung robot sejajar dengan path.

Dinonaktifkan jika robot sudah dekat dengan goal:

$$\text{disabled if } \|p_{\text{robot}} - p_{\text{goal}}\| < d_{\text{forward}} \cdot \text{cheat_factor}$$

Seleksi Trajectory terbaik

```

1 1. Prepare semua critics:
2   - MapGridCostFunction::prepare() -> wavefront BFS propagation
3 2. Iterate semua trajectory dari generator:
4   a. GenerateTrajectory(pos, vel, sample_vel, &traj)
5   b. scoreTrajectory(traj, best_cost)
```

6 c. Jika $\text{cost} \geq 0$ dan $\text{cost} < \text{best_cost}$ -> update best trajectory
7 3. Jika tidak ada trajectory valid -> gunakan zero velocity

Early termination: Jika total cost sudah melebihi best_cost saat menjumlahkan critics, evaluasi trajectory dihentikan lebih awal.

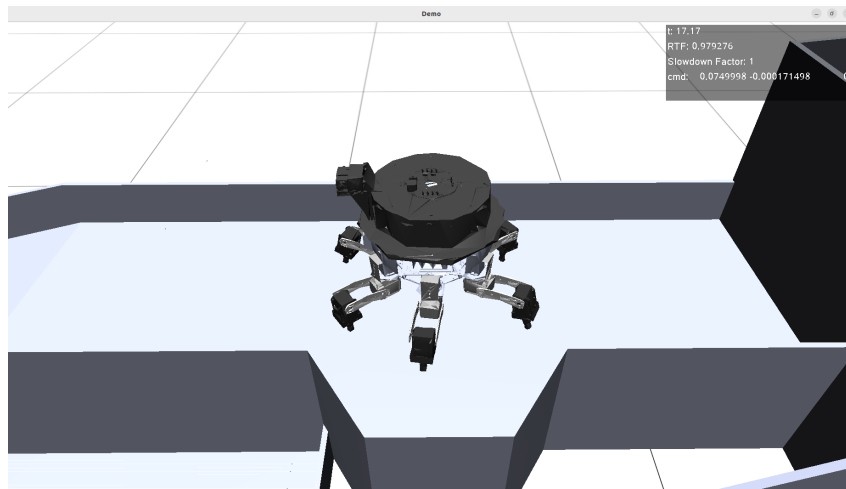
[Halaman ini sengaja dikosongkan]

BAB IV

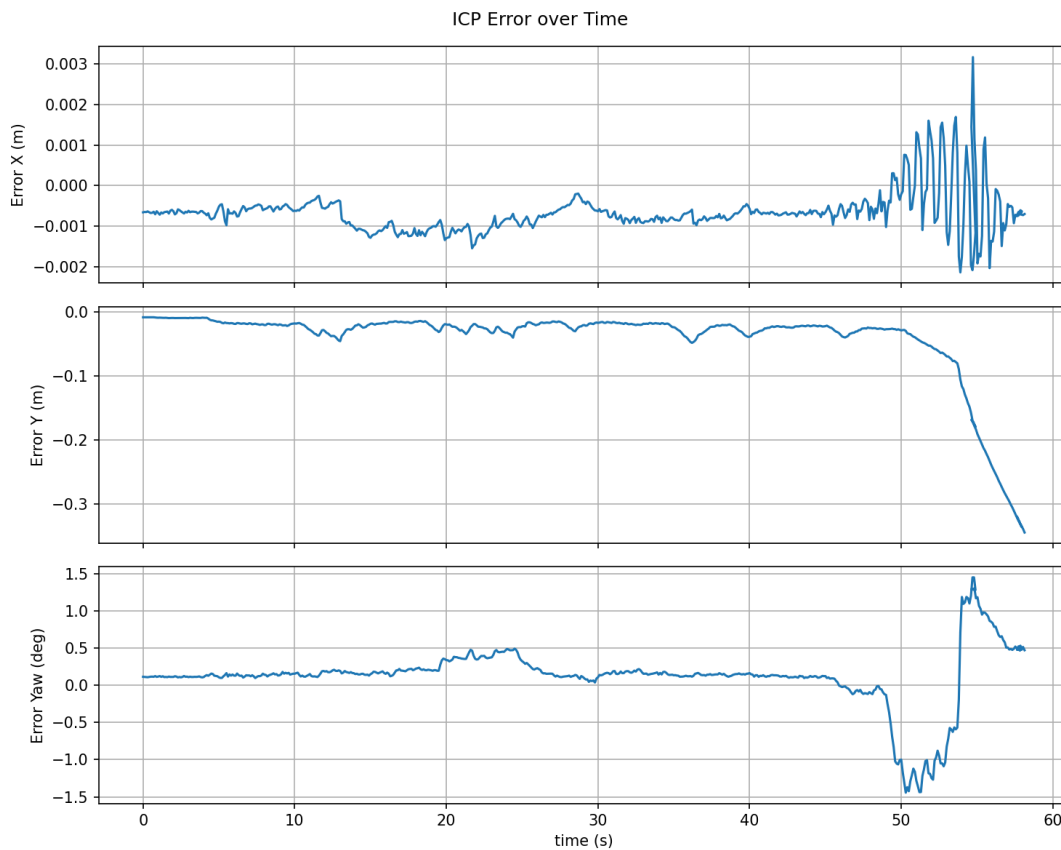
HASIL DAN PEMBAHASAN

4.1 Hasil dan Analisis

4.1.1 Hasil Lokalisasi ICP

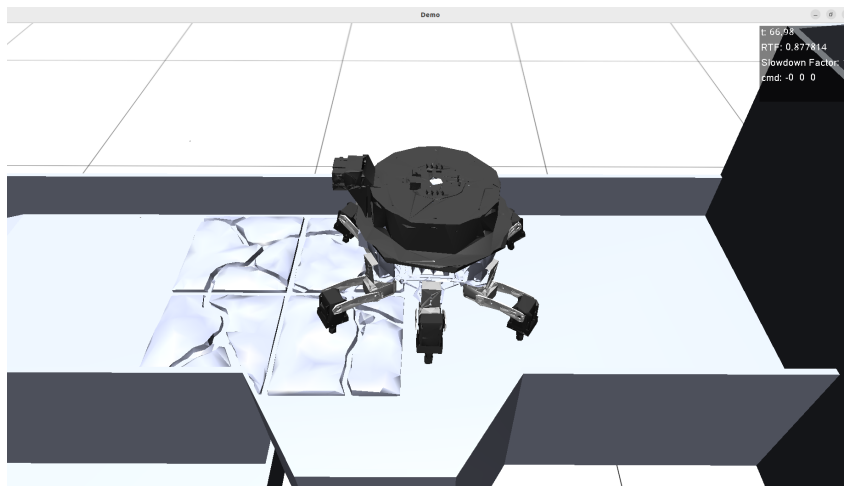


Gambar 4.1: Simulasi di Region A

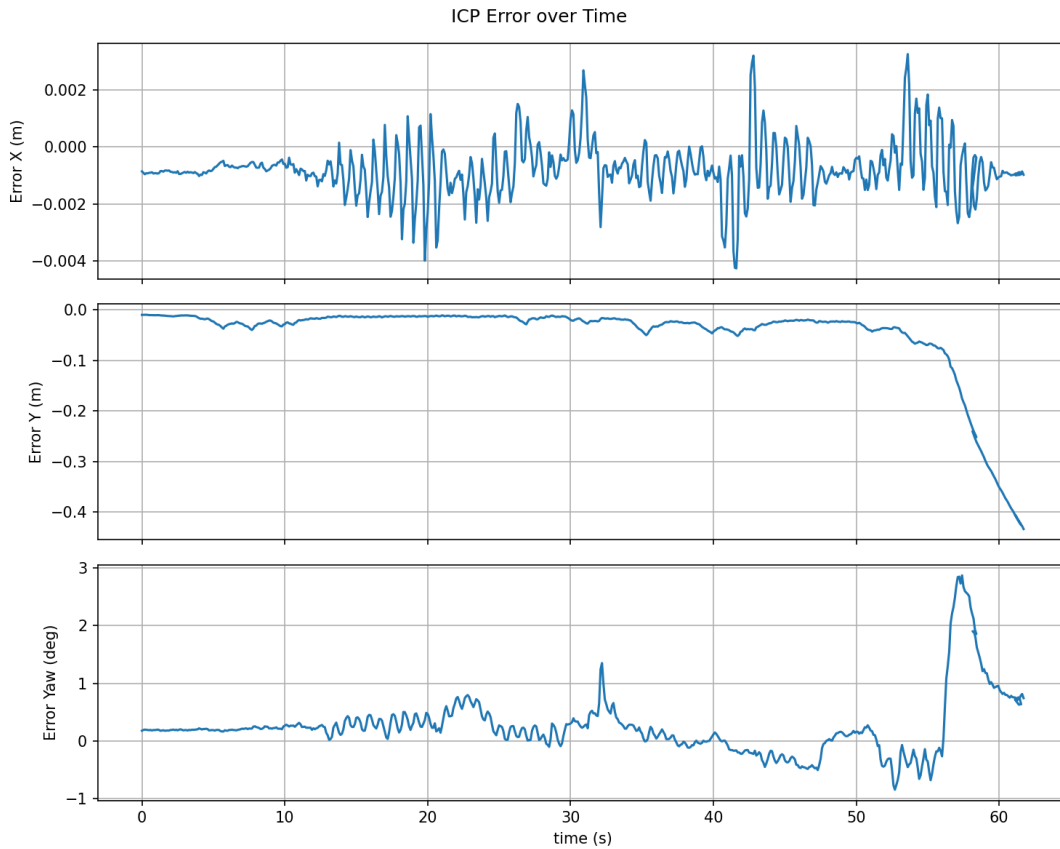


Gambar 4.2: Error ICP di Region A

Grafik pada Gambar 4.2 menunjukkan error ICP pada sumbu x , y , dan θ (yaw) saat robot melintasi Region A. Secara umum, performa ICP stabil dengan tingkat error yang rendah sepanjang lintasan. Namun, terjadi lonjakan error yang signifikan pada sekitar $t = 50$ detik. Lonjakan ini disebabkan oleh posisi robot yang berada di zona transisi antara dua *region*, di mana lingkungan yang dapat terdeteksi oleh sensor LiDAR sangat terbatas. Akibatnya, kualitas *scan matching* menurun dan menghasilkan akumulasi error yang lebih besar pada estimasi transformasi.

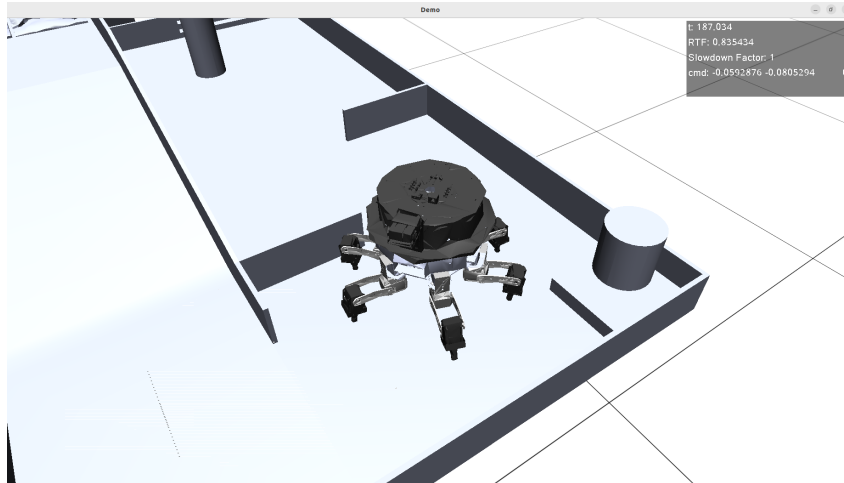


Gambar 4.3: Simulasi di Region A dengan rintangan lantai

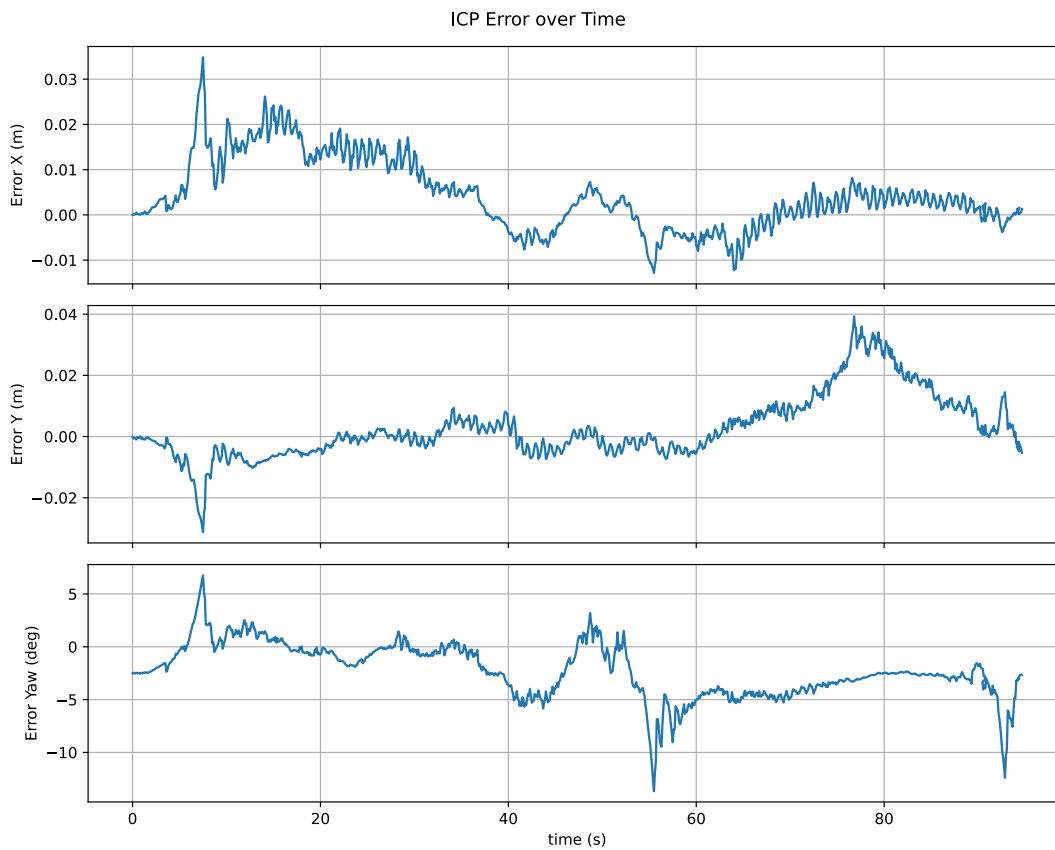


Gambar 4.4: Error ICP di Region A dengan rintangan lantai

Selanjutnya, performa ICP diuji pada region A dengan kondisi lantai yang tidak rata untuk mensimulasikan skenario lantai yang rusak akibat gempa bumi. Gambar 4.4 menunjukkan error ICP pada sumbu x , y , dan θ (yaw) untuk skenario ini. Dibandingkan dengan kasus sebelumnya, terjadi peningkatan osilasi yang signifikan, terutama pada sumbu x dan yaw. Hal ini disebabkan oleh permukaan lantai yang tidak rata sehingga menghasilkan titik-titik *scan* LiDAR yang lebih bervariasi secara vertikal, yang menurunkan kualitas *scan matching* pada proses ICP. Akibatnya, estimasi transformasi menjadi kurang stabil dan menghasilkan fluktuasi error yang lebih besar. Lonjakan error di sekitar $t = 55$ detik tetap terjadi karena faktor zona transisi antar *region*.



Gambar 4.5: Simulasi di Region C

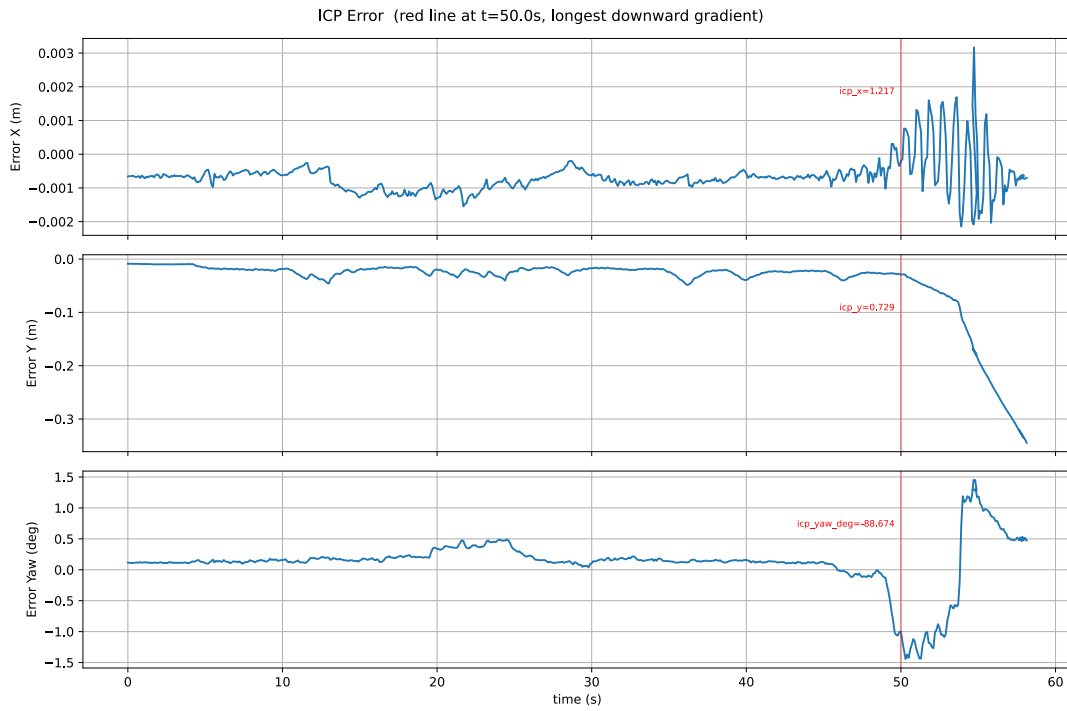


Gambar 4.6: Error ICP di Region C

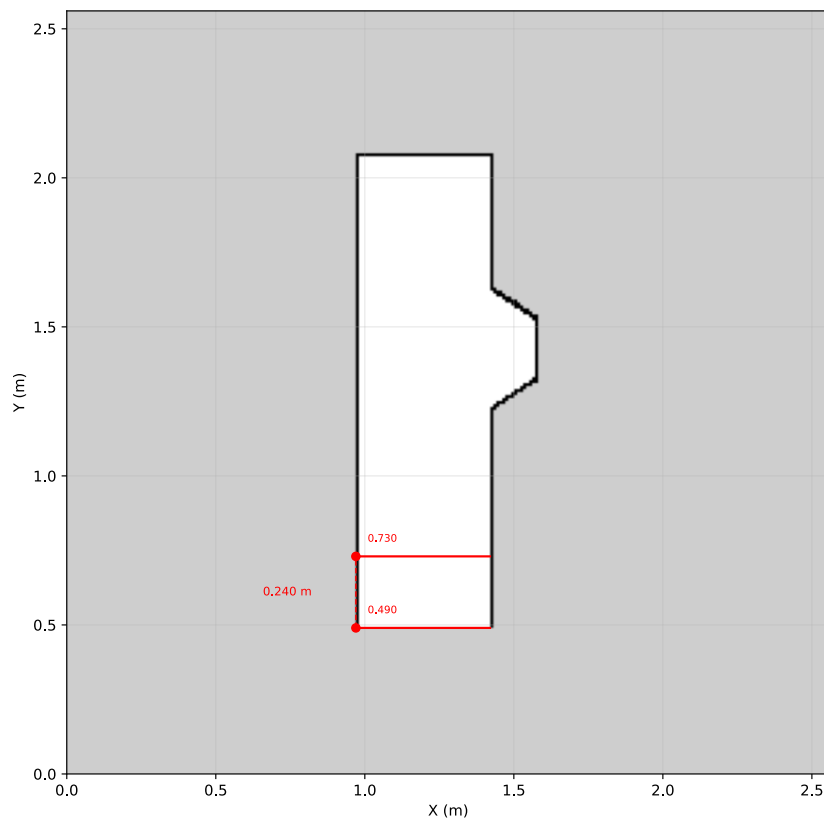
Selanjutnya, performa ICP diuji pada Region C. Gambar 4.6 menunjukkan error ICP pada sumbu x , y , dan θ (yaw) untuk skenario ini. Hasilnya menunjukkan bahwa error maksimum pada ketiga sumbu lebih tinggi dibandingkan dengan performa di Region A, dan secara umum error yang dihasilkan lebih tidak stabil. Hal ini disebabkan oleh dua faktor. Pertama, Region C memiliki jumlah sel *obstacle* yang lebih banyak, sehingga tabel *hash KNN* yang perlu dicari lebih besar dan berpotensi memperlambat serta menurunkan kualitas *scan matching*. Kedua,

trajectory robot di Region C lebih kompleks dengan banyak rotasi, yang mengakibatkan akumulasi error yang lebih besar pada estimasi *yaw*.

4.1.2 Hasil Zona Transisi



Gambar 4.7: Error ICP di Region A dengan data posisi



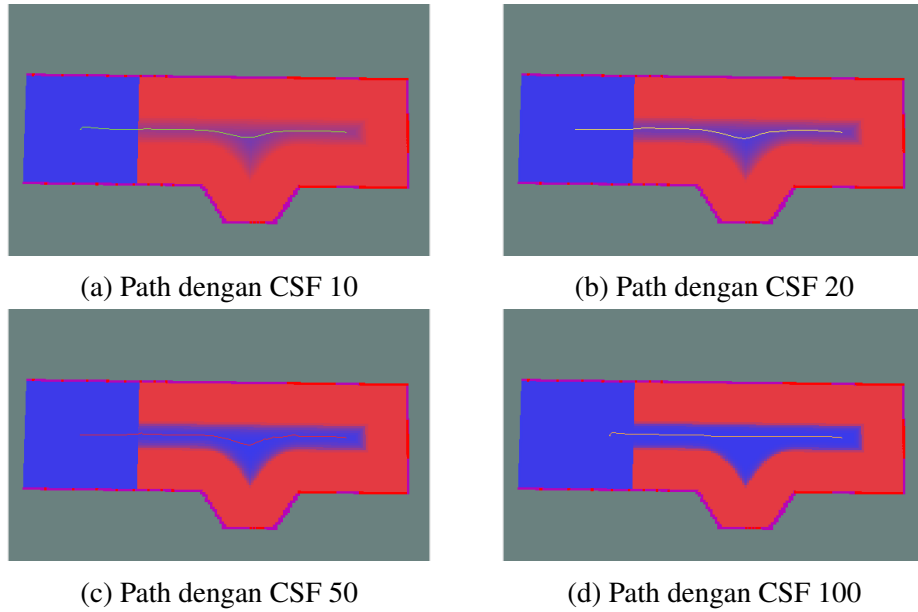
Gambar 4.8: Radius zona transisi

Pada data pada grafik 4.2, bisa juga diketahui posisi robot saat kualitas ICP menurun. dari data tersebut dapat dilihat bahwa kualitas ICP turun mulai saat robot posisi $y=0.729$ m, atau saat robot berada 0.24 dari ujung map. Nilai tersebut menunjukkan bahwa zona transisi cocok untuk diberi nilai radius minimal 0.24, untuk bisa transisi dengan aman

Berdasarkan Gambar 4.2, dapat pula diidentifikasi posisi robot saat kualitas ICP mulai menurun. Dari data tersebut, penurunan kualitas ICP mulai terjadi ketika robot berada pada posisi $y = 0.729$ m, atau sekitar 0.24 m dari ujung *map*. Hal ini menunjukkan bahwa zona transisi perlu dirancang dengan radius minimal 0.24 m agar robot dapat melakukan transisi antar *region* dengan aman sebelum kualitas lokalisasi mengalami penurunan signifikan.

4.1.3 Hasil Global Planner

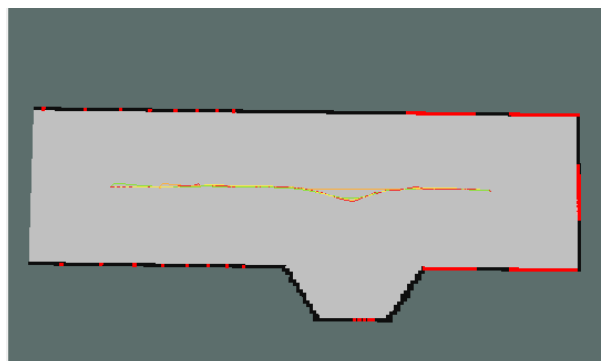
4.1.3.1 Hasil Path di Region A



Gambar 4.9: Relasi path terhadap Cost Scaling Factor di Region A

Cost Scaling Factor	Panjang path (m)	Waktu komputasi (ms)	Min clearance (m)	Jumlah waypoint
10	1.116051	6.666027	0.220000	222
20	1.114836	6.897399	0.220000	223
50	1.122597	5.688345	0.219650	225
100	0.971836	6.484874	0.216470	193

Tabel 4.1: Karakteristik Path di Region A berdasarkan Cost Scaling Factor



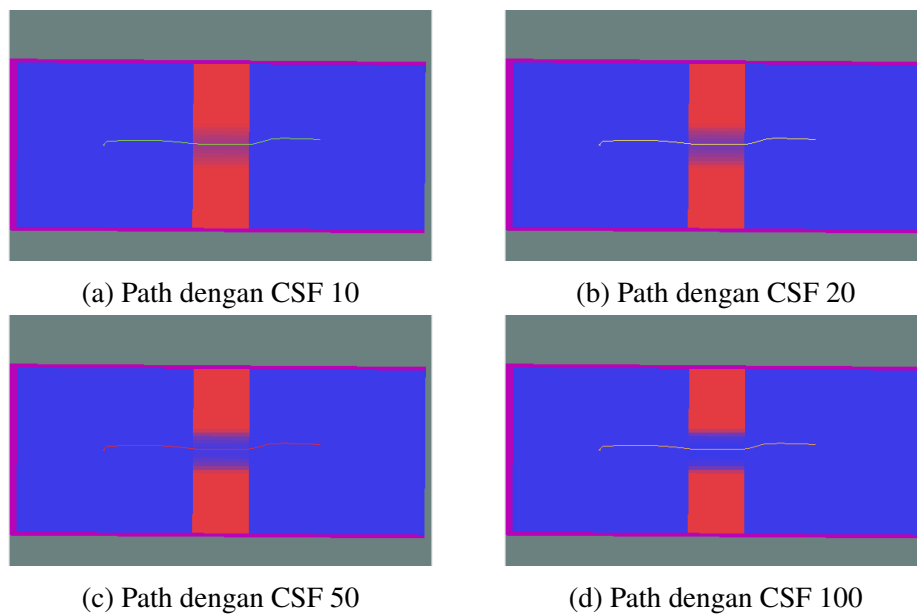
Gambar 4.10: Semua path di region A.
 ● CSF 10, ● CSF 20, ● CSF 50, ● CSF 100

Global path adalah jalur yang direncanakan dari posisi robot menuju suatu tujuan. Pada pengujian di Region A, tujuan yang diberikan adalah titik tengah zona transisi yang ditandai

dengan area biru di sebelah kiri peta, yang berfungsi sebagai gerbang menuju *region* berikutnya. Pengujian dilakukan dengan berbagai nilai *Cost Scaling Factor* (CSF) untuk melihat pengaruhnya terhadap bentuk *path* yang dihasilkan. Pengaruh CSF terhadap *global costmap* dijelaskan pada persamaan (3.23), yakni semakin tinggi nilai CSF, semakin curam penurunan nilai biaya terhadap jarak dari *obstacle*.

Hasil *path* untuk setiap nilai CSF ditunjukkan pada Gambar 4.9 dan karakteristiknya dirangkum pada Tabel 4.1. Pada CSF 10, *path* cenderung mengikuti titik tengah di antara dua *obstacle*. Hal ini menghasilkan *path* yang tidak optimal karena melakukan belokan yang tidak diperlukan dan memiliki panjang yang bukan terpendek. Sebaliknya, pada CSF 100, *path* bergerak lebih mepet ke *obstacle*, memanfaatkan area biaya rendah yang lebih luas di dekat dinding. Hasilnya adalah *path* yang lebih lurus dan lebih pendek.

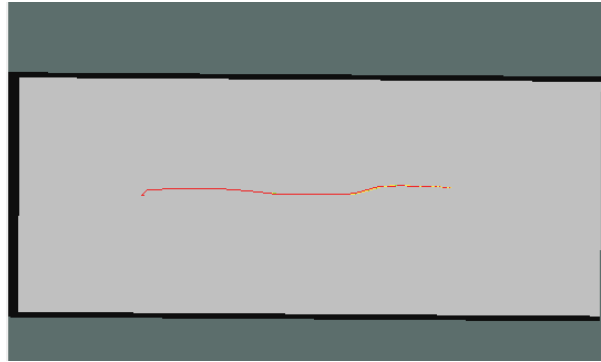
4.1.3.2 Hasil Path di Region B



Gambar 4.11: Relasi path terhadap Cost Scaling Factor di Region B

Cost Scaling Factor	Panjang path (m)	Waktu komputasi (ms)	Min clearance (m)	Jumlah waypoint
10	0.591730	152.256312	0.208163	118
20	0.591773	135.818868	0.208066	118
50	0.591600	20.557631	0.208224	118
100	0.591590	10.574465	0.208132	118

Tabel 4.2: Karakteristik Path di Region B berdasarkan Cost Scaling Factor



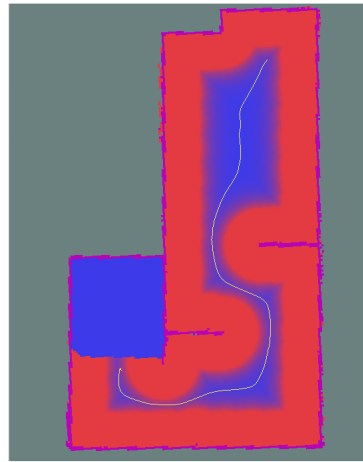
Gambar 4.12: Semua path di region B.
● CSF 10, ● CSF 20, ● CSF 50, ● CSF 100

Hasil *path* untuk setiap nilai CSF di Region B ditunjukkan pada Gambar 4.11 dan dilengkapi data kuantitatif pada Tabel 4.2. Seluruh *path* dimulai dari posisi robot setelah bertransisi dari Region A ke Region B dan berakhir di zona transisi menuju Region C. Perbedaan nilai CSF tidak memberikan pengaruh yang signifikan terhadap bentuk *path* yang dihasilkan. Hal ini disebabkan oleh *layout* Region B yang hanya berupa lorong sempit, sehingga tidak terdapat ruang terbuka yang cukup bagi variasi *cost gradient* untuk mengubah lintasan *path* secara berarti. Akibatnya, semua *path* dari keempat nilai CSF hampir identik satu sama lain, sebagaimana terlihat pada Gambar 4.12.

4.1.3.3 Hasil Path di Region C



(a) Path dengan CSF 10



(b) Path dengan CSF 20



(c) Path dengan CSF 50

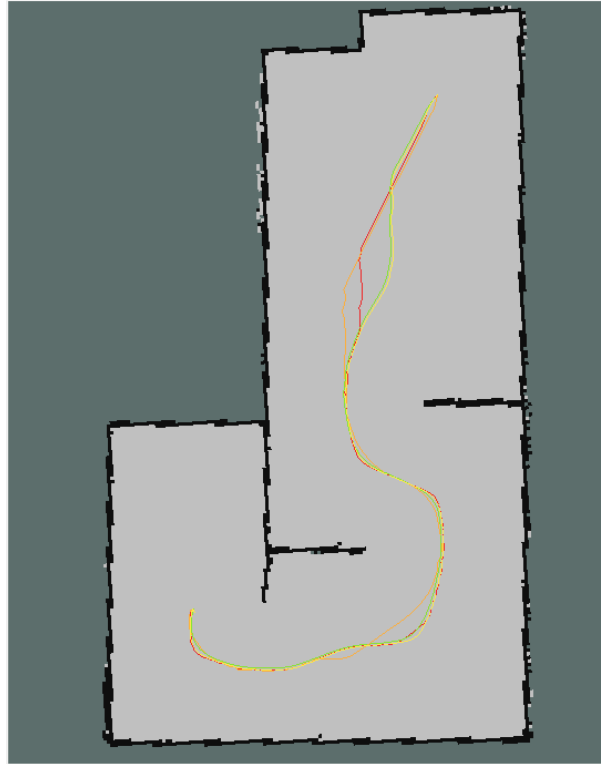


(d) Path dengan CSF 100

Gambar 4.13: Relasi path terhadap Cost Scaling Factor di Region C

Cost Scaling Factor	Panjang path (m)	Waktu komputasi (ms)	Min clearance (m)	Jumlah waypoint
10	2.542122	10.629525	0.197048	509
20	2.596590	10.268101	0.196292	521
50	2.596873	9.922248	0.197185	520
100	2.524643	10.224478	0.195499	506

Tabel 4.3: Karakteristik Path di Region C berdasarkan Cost Scaling Factor



Gambar 4.14: Semua path di region C.
 ● CSF 10, ● CSF 20, ● CSF 50, ● CSF 100

Hasil *path* untuk setiap nilai CSF di Region C ditunjukkan pada Gambar 4.13 dan dilengkapi data kuantitatif pada Tabel 4.3. Pada CSF 10, *path* cenderung mempertahankan posisi di tengah lorong pada semua kondisi, yang menghasilkan jalur yang lebih aman namun lebih panjang. Sebaliknya, pada CSF 100, *path* tidak terikat pada posisi tengah, sehingga mampu memanfaatkan ruang area terbuka, dan menghasilkan *path* yang lebih pendek dan lurus. *Path* pada CSF 20 dan CSF 50 menunjukkan karakteristik peralihan di antara kedua ekstrem tersebut. Dari keseluruhan hasil, CSF 100 menghasilkan *path* yang paling sesuai untuk navigasi di Region C karena memberikan keseimbangan terbaik antara efisiensi jalur (lebih pendek) dan *clearance* terhadap *obstacle* yang masih memadai.

Cost Scaling Factor	Mean cross track error (m)	Max cross track error (m)	Standard deviation (m)
10	0.006029	0.025937	0.006468
20	0.006880	0.029719	0.007214
50	0.006536	0.033099	0.007537
100	0.003537	0.010000	0.002203

Tabel 4.4: Perbandingan path terhadap ground truth di region A

Cost Scaling Factor	Mean cross track error (m)	Max cross track error (m)	Standard deviation (m)
10	0.005756	0.011590	0.003508
20	0.005778	0.011591	0.003492
50	0.005678	0.011591	0.003483
100	0.005678	0.011591	0.003483

Tabel 4.5: Perbandingan path terhadap ground truth di region B

Cost Scaling Factor	Mean cross track error (m)	Max cross track error (m)	Standard deviation (m)
20	0.005548	0.002228	0.013089
50	0.012802	0.018897	0.089070
100	0.020026	0.024742	0.099313

Tabel 4.6: Perbandingan path terhadap ground truth di region C

4.1.4 Hasil DWA Planner

Region	Waktu (s)	Mean cross track error (m)	Max cross track error (m)
A	13.412	0.009814	0.033543
B	7.707	0.041840	0.029113
C	96.141	0.022437	0.086539

Tabel 4.7: Hasil tracking path

Region	Waktu (s)	Mean cross track error (m)	Max cross track error (m)
A	13.144	0.004394	0.026285
B	7.243	0.013691	0.064845
C	94.119	0.039834	0.174556

Tabel 4.8: Hasil tracking path kondisi gempu

Pengujian *DWA planner* bertujuan untuk mengevaluasi kemampuannya dalam mengikuti *global path* yang telah direncanakan sebelumnya. Hasil *tracking* untuk kondisi normal dan kondisi gempu masing-masing dirangkum pada Tabel 4.7 dan Tabel 4.8. Pada kondisi normal, *DWA planner* dapat mengikuti *global path* dengan baik, tercermin dari nilai *mean cross track error* dan *max cross track error* yang relatif rendah. Sebaliknya, pada kondisi gempu, nilai *max cross track error* meningkat secara signifikan. Hal ini terjadi karena terdapat *obstacle* yang tidak diketahui oleh *global path*, namun terdeteksi oleh sensor LiDAR saat navigasi berlangsung. Ketika *obstacle* tersebut berada tepat pada lintasan *global path*, *DWA planner* terpaksa menghasilkan *trajectory* yang menyimpang menjauhi *global path* untuk menghindari tabrakan. Simpangan inilah yang tercermin sebagai nilai *max cross track error* yang lebih besar.

4.2 Pembahasan

[Halaman ini sengaja dikosongkan]

BAB V

PENUTUP

5.1 Kesimpulan

5.2 Saran

[Halaman ini sengaja dikosongkan]

DAFTAR PUSTAKA

- Damle, V. P., & Susan, S. (2022). Dynamic algorithm for path planning using a-star with distance constraint. *2022 2nd International Conference on Intelligent Technologies (CONIT)*, 1–5.
- Hu, B., Cui, M., & Cao, Z. (2022). A slope-adaptive navigation approach for ground mobile robots. *2022 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 610–615.
- Jung, D., Kim, C., Cho, J.-K., & Kim, S.-W. (2024). Munes: Multifloor navigation including elevators and stairs. *arXiv preprint arXiv:2402.04535*.
- Kim, E.-H., Bae, S.-H., & Kuc, T.-Y. (2020). Mobile service robot multi-floor navigation using visual detection and recognition of elevator features (iccas 2020). *2020 20th International Conference on Control, Automation and Systems (ICCAS)*, 982–985.
- Kim, T., Kang, G., Lee, D., & Shim, D. H. (2024). Development of an indoor delivery mobile robot for a multi-floor environment. *IEEE Access*.
- Lee, J. Y., Tokuda, T., & Sato, K. (2023). Autonomous floor navigation control of mobile robot using elevator button recognition with deep learning. *2023 62nd Annual Conference of the Society of Instrument and Control Engineers (SICE)*, 133–138.
- Li, S., Chen, Y., Meng, Y., & Lou, Y. (2021). Autonomous elevator button recognition and operation framework for multi-floor mobile manipulator navigation. *2021 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*, 383–388.
- Lin, T.-H., Huang, J.-T., & Putranto, A. (2022). Integrated smart robot with earthquake early warning system for automated inspection and emergency response. *Natural hazards*, 110(1), 765–786.
- Macenski, S., Moore, T., Lu, D. V., Merzlyakov, A., & Ferguson, M. (2023). From the desks of ros maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2. *Robotics and Autonomous Systems*, 168, 104493.
- Palacín, J., Bitriá, R., Rubies, E., & Clotet, E. (2023). A procedure for taking a remotely controlled elevator with an autonomous mobile robot based on 2d lidar. *Sensors*, 23(13), 6089.
- Palacín, J., Rubies, E., Bitriá, R., & Clotet, E. (2023). Path planning of a mobile delivery robot operating in a multi-story building based on a predefined navigation tree. *Sensors*, 23(21), 8795.
- Sabtaji, A. (2020). Statistik kejadian gempa bumi tektonik tiap provinsi di wilayah indonesia selama 11 tahun pengamatan (2009-2019). *Buletin Meteorologi, Klimatologi, dan Geofisika*, 1(7), 31–46.
- Sobreira, H., Costa, C. M., Sousa, I., Rocha, L., Lima, J., Farias, P., Costa, P., & Moreira, A. P. (2019). Map-matching algorithms for robot self-localization: A comparison between perfect match, iterative closest point and normal distributions transform. *Journal of Intelligent & Robotic Systems*, 93, 533–546.
- Xie, H., & Zhou, Q. (2023). Autonomous traversing across floors based on vision for reconfigurable tracked robot. *2023 8th International Conference on Control, Robotics and Cybernetics (CRC)*, 14–19.

- Yao, Q., Tian, Y., Wang, Q., & Wang, S. (2020). Control strategies on path tracking for autonomous vehicle: State of the art and future challenges. *IEEE Access*, 8, 161211–161222.
- Zhang, Y., Li, Y., Zhang, H., Wang, Y., Wang, Z., Ye, Y., Yue, Y., Guo, N., Gao, W., Chen, H., et al. (2023). Earthshaker: A mobile rescue robot for emergencies and disasters through teleoperation and autonomous navigation. *JUSTC*, 53(1), 4–1.
- Zhu, K., Zhan, J., Chen, S., Nan, Z., Zhang, T., Zhu, D., & Zheng, N. (2020). Atv navigation in complex and unstructured environment containing stairs. *2020 IEEE Intelligent Vehicles Symposium (IV)*, 817–824.

LAMPIRAN

ddd

[Halaman ini sengaja dikosongkan]

BIOGRAFI PENULIS

[Halaman ini sengaja dikosongkan]